TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



TĂNG DUY VIÊT – 52400255
LƯU NHẬT QUANG – 52400229

XÂY DỰNG WEBSITE KHÁM PHÁ QUÁN ĂN "WHAT2EAT"

BÁO CÁO KẾT THÚC MÔN LẬP TRÌNH WEB VÀ ỨNG DỤNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



TĂNG DUY VIỆT – 52400255
LƯU NHẬT QUANG – 52400229

XÂY DỰNG WEBSITE KHÁM PHÁ QUÁN ĂN "WHAT2EAT"

BÁO CÁO KẾT THÚC MÔN LẬP TRÌNH WEB VÀ ỨNG DỤNG

Người hướng dẫn
ThS. Doãn Xuân Thanh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Nhóm chúng em xin cảm ơn Quý Thầy Cô Khoa Công nghệ thông tin, Trường Đại học Tôn Đức Thắng, đã truyền đạt kiến thức nền tảng về lập trình web, cơ sở dữ liệu và phân tích thiết kế hệ thống.

Đó là hành trang để nhóm tiếp cận và hoàn thành đồ án.

Nhóm xin cảm ơn giảng viên hướng dẫn ThS. Doãn Xuân Thanh đã góp ý về phạm vi đề tài, cấu trúc báo cáo và cách trình bày sản phẩm.

Các nhận xét trong những lần báo cáo tiến độ giúp nhóm phát hiện sớm chỗ chưa thống nhất giữa mô tả đề tài và kết quả thực tế, từ đó kịp điều chỉnh hướng triển khai.

Chúng em cũng cảm ơn các bạn sinh viên đã dùng thử phiên bản nội bộ, gửi phản hồi về giao diện và bộ lọc. Nhờ dữ liệu thật của OpenStreetMap, nhóm có cơ sở kiểm chứng lựa chọn kỹ thuật thay vì chỉ dựa trên giả định.

TP. Hồ Chí Minh, ngày 10 tháng 11 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

CAM ĐOAN

Nhóm xin cam đoan toàn bộ mã nguồn, số liệu và kết quả trong báo cáo là sản phẩm do nhóm thực hiện, dưới sự hướng dẫn của giảng viên.

Mọi nội dung tham khảo đều được trích dẫn rõ trong phần Tài liệu tham khảo.

Toàn bộ thông tin kỹ thuật được đối chiếu trực tiếp với mã nguồn tại thời điểm viết, kèm vị trí file và số dòng.

Nhận định nào chưa có bằng chứng trong mã nguồn đều được đánh dấu là suy luận hoặc nêu rõ là hạn chế mở.

Nếu phát hiện bất kỳ sự gian lận nào, chúng em xin hoàn toàn chịu trách nhiệm về nội dung đồ án của mình.

TP. Hồ Chí Minh, ngày 10 tháng 11 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

XÂY DỰNG WEBSITE KHÁM PHÁ QUÁN ĂN VÀ HỖ TRỢ GHI CHÉP NHẬT KÝ ẨM THỰC CÁ NHÂN "WHAT2EAT"

TÓM TẮT

What2Eat trả lời câu hỏi "hôm nay ăn gì?" của sinh viên và người trẻ tại Việt Nam.

Hệ thống gợi ý quán ăn theo danh mục, ngân sách và khoảng cách; ghi nhật ký ẩm thực kèm ảnh và nhãn; lên kế hoạch ăn nhóm có bình chọn; thống kê thói quen chi tiêu.

Sản phẩm viết bằng PHP 8 thuần theo mô hình REST API một cửa, cơ sở dữ liệu MySQL 8 với 23 bảng.

Giao diện phía trình duyệt dùng HTML, CSS, JavaScript kết hợp Leaflet và Chart.js.

Dữ liệu quán ăn lấy từ OpenStreetMap qua Overpass API, kèm bộ đệm và ba tầng dự phòng khi dịch vụ ngoài chậm hoặc lỗi.

Đăng nhập gồm email – mật khẩu (Argon2id) và đăng nhập Google, trong đó chữ ký JWT RS256 được tự kiểm tra.

Kết quả: hệ thống chạy end-to-end với 20 trang giao diện, 56 đường dẫn API và dữ liệu 8.600 địa điểm TP. Hồ Chí Minh.

Bộ kiểm thử tích hợp tự tạo – xoá cơ sở dữ liệu tạm và kiểm tra 12 nhóm chức năng.

Các hạn chế còn mở: chưa có CSP, chưa có xác thực hai bước, phụ thuộc dịch vụ OSM công cộng.

Từ khoá: PHP, REST API, MySQL, OpenStreetMap, Overpass, PWA, nhật ký ẩm thực, bình chọn nhóm.

BUILDING A RESTAURANT EXPLORATION AND PERSONAL FOOD DIARY WEB APPLICATION: "WHAT2EAT"

ABSTRACT

What2Eat answers the everyday question "what should I eat today?" for students and young users in Vietnam.

It recommends restaurants by cuisine, budget and distance, keeps a personal food diary with photos and tags, supports group meal plans with voting, and shows spending statistics.

The product is a framework-free PHP 8 REST API with a single routing entry point and a 23-table MySQL 8 database.

The front end uses HTML, CSS and vanilla JavaScript together with Leaflet and Chart.js.

Restaurant data comes from OpenStreetMap through the Overpass API, backed by a cache and three fallback layers.

Authentication covers Argon2id passwords and Google Sign-In, where the ID token signature is verified by our own RS256 code.

The system runs end to end: 20 interface pages, 56 API routes and 8,600 mapped places in Ho Chi Minh City.

An integration suite creates a temporary database and exercises twelve functional groups.

Open limitations are stated: no Content-Security-Policy header, no two-factor authentication, and dependence on public OSM services.

Keywords: PHP, REST API, MySQL, OpenStreetMap, Overpass, PWA, food diary, group voting.

MỤC LỤC

DANH MỤC HÌNH VẼ ix

DANH MỤC BẢNG BIỂU xiii

DANH MỤC CÁC CHỮ VIẾT TẮT xiv

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI 1

1.1 Lý do chọn đề tài 1

1.2 Mục tiêu của đề tài 1

1.2.1 Mục tiêu chung 1

1.2.2 Mục tiêu cụ thể 1

1.3 Phạm vi và đối tượng sử dụng 2

1.4 Phương pháp thực hiện 2

1.5 Bố cục báo cáo 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH HỆ THỐNG 3

2.1 Công nghệ sử dụng 3

2.1.1 PHP 8 theo mô hình API không framework 3

2.1.2 MySQL 8 và PDO 4

2.1.3 Frontend: HTML, CSS, JavaScript thuần, Bootstrap, Leaflet, Chart.js 5

2.1.4 OpenStreetMap, Overpass API và Nominatim 5

2.1.5 Google Identity Services 5

2.1.6 Progressive Web App 6

2.1.7 Tổng hợp công nghệ 6

2.2 Kiến trúc hệ thống 7

2.2.1 Ba lớp và một cửa vào 7

2.2.2 Phân tách trách nhiệm trong api/ *8*

2.2.3 Bề mặt API *9*

2.3 Phân tích yêu cầu 9

2.3.1 Yêu cầu chức năng *9*

2.3.2 Yêu cầu phi chức năng *10*

2.3.3 Phân quyền *11*

2.4 Thiết kế cơ sở dữ liệu 12

2.4.1 Quy ước *12*

2.4.2 Sơ đồ thực thể – quan hệ *12*

2.4.3 Danh mục bảng *14*

2.4.4 Ràng buộc toàn vẹn *16*

2.5 Luồng dữ liệu địa điểm OpenStreetMap 16

2.6 Đánh giá lựa chọn kiến trúc 18

CHƯƠNG 3. GIAO DIỆN VÀ QUY TRÌNH SỬ DỤNG 18

3.1 Trang chủ và phiên khách 18

3.2 Đăng ký, đăng nhập và xác minh email 21

3.3 Khám phá quán ăn theo vị trí 23

3.4 Nhật ký ẩm thực cá nhân 27

3.5 Kế hoạch ăn uống theo nhóm 30

3.6 Thống kê cá nhân và quản trị tài khoản 33

3.7 Khu vực quản trị 36

3.8 Kết luận chương 3 39

CHƯƠNG 4. KỸ THUẬT TRIỂN KHAI VÀ LUỒNG XỬ LÝ 40

4.1 Khởi động và đường đi của một yêu cầu 40
4.1.1 Một cửa vào duy nhất 40
4.1.2 Lớp bootstrap và kết nối cơ sở dữ liệu 41
4.1.3 Luồng đầy đủ của một yêu cầu có xác thực 41
4.2 Xác thực, phân quyền và bảo vệ phiên 42
4.2.1 Mật khẩu và luồng đăng ký 42
4.2.2 Phiên làm việc và thu hồi hàng loạt 43
4.2.3 Chống CSRF 44
4.2.4 Giới hạn tần suất 45
4.2.5 Đăng nhập Google và việc tự viết bộ kiểm tra JWT 46
4.2.6 Ma trận phân quyền 48
4.3 Khai thác dữ liệu không gian OpenStreetMap 48
4.3.1 Câu truy vấn Overpass 49
4.3.2 Đua ba máy chủ Overpass 49
4.3.3 Bốn tầng dự phòng 51
4.3.4 Proxy tile và tôn trọng chính sách sử dụng 51
4.3.5 Nominatim và khoá phân tán trong MySQL 52
4.3.6 Lọc theo khoảng cách: bounding box trước, Haversine sau 53
4.3.7 Tìm kiếm không phân biệt dấu 54
4.3.8 Bộ dữ liệu nền 8.600 địa điểm 55
4.4 Vòng đời một nhật ký ẩm thực 56
4.5 Kế hoạch nhóm: giao dịch, khoá và bình chọn 57
4.6 Quản lý tệp tin và phân quyền truy cập 58

4.7 PWA, ngoại tuyến và tối ưu tải trang.....59

4.8 Phía trình duyệt: tổ chức mã không dùng bundler.....61

4.9 Vận hành: migration, công cụ dòng lệnh và lịch chạy định kỳ.....61

4.10 Triển khai.....62

4.11 Kiểm thử.....64

4.12 Kết luận chương 4.....64

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....66

5.1 Kết quả đạt được so với mục tiêu.....66

5.2 Hạn chế còn tồn tại.....67

5.3 Kết luận chung.....67

TÀI LIỆU THAM KHẢO.....69

Tài liệu tiếng Việt và tài liệu nội bộ.....69

Tài liệu tiếng Anh.....69

DANH MỤC HÌNH VẼ

Hình 2.1: Hàm json_response(): đường ra JSON chuẩn của toàn hệ thống 4

Hình 2.2: Kiến trúc tổng thể của What2Eat 7

Hình 2.3: Bảy quy tắc rewrite trong tệp .htaccess ở gốc dự án.....8

Hình 2.4: ERD nhóm người dùng, nhật ký ẩm thực, nhãn và media.....13

Hình 2.5: ERD nhóm danh mục, quán ăn, tài trợ và đề xuất 13

Hình 2.6: ERD nhóm kế hoạch ăn và bình chọn.....14

Hình 2.7: ERD nhóm dữ liệu địa điểm OSM và vận hành 14

Hình 2.8: Khoá, index và ràng buộc CHECK của bảng food_notes.....16

Hình 2.9: Luồng xử lý và các tầng dự phòng khi lấy dữ liệu quán ăn.....17

Hình 3.1: Đầu trang chủ: định vị thương hiệu và các gợi ý nhanh 19

Hình 3.2: Giữa trang chủ: danh sách địa điểm nổi bật và khối giới thiệu tính năng 20

Hình 3.3: Cuối trang chủ: khu vực Food Plan nhóm và footer.....20

Hình 3.4: Màn hình "Tạo tài khoản mới" với biểu thức chính quy kiểm độ mạnh mật khẩu 21

Hình 3.5: Màn hình "Chào mừng trở lại" – đăng nhập bằng email và mật khẩu.....22

Hình 3.6: Nút "Tiếp tục với Google" và hộp đồng ý của Google Identity Services.22

Hình 3.7: Bản đồ khám phá với các marker địa điểm và lời chú dẫn OpenStreetMap 23

Hình 3.8: Thanh bộ lọc: Khoảng cách, Mức giá, Đánh giá và thứ tự sắp xếp.....24

Hình 3.9: Danh sách địa điểm quét được trong khu vực đang xem.....24

Hình 3.10: Hộp thoại "Chọn địa điểm ngẫu nhiên" khi người dùng không biết ăn gì 25

Hình 3.11: Xác định vị trí hiện tại bằng Geolocation API của trình duyệt.....25

Hình 3.12: Chi tiết một địa điểm: thông tin, đánh giá, nhật ký ẩm thực liên quan...26

Hình 3.13: Biểu mẫu "Bổ sung thông tin" cho một địa điểm 26

Hình 3.14: Biểu mẫu "Báo thông tin sai".....27

Hình 3.15: Trang "Food Note Của Tôi" với bộ đếm tổng, ghim, yêu thích và điểm trung bình 28

Hình 3.16: Bước 1 của biểu mẫu tạo ghi chú: tên món, ngày ăn, giá và đánh giá....28

Hình 3.17: Bước 2: nhận xét, nhãn và tải ảnh minh hoạ cho món ăn 29

Hình 3.18: Thêm địa điểm mới ngay trong biểu mẫu ghi chú 29

Hình 3.19: Danh sách "Kế hoạch của bạn" – cả kế hoạch đang mở lẫn đã quyết định 30

Hình 3.20: Trang tạo kế hoạch: tên, mô tả và thời gian hẹn 31

Hình 3.21: Hộp thoại xác nhận thông tin kế hoạch trước khi gửi.....31

Hình 3.22: Màn hình chọn địa điểm đề xuất cho kế hoạch.....32

Hình 3.23: Thẻ tổng hợp mỗi kế hoạch: trạng thái, số thành viên, số địa điểm đề xuất và ngày hẹn.....32

Hình 3.24: Bảng điều khiển: tổng chi tiêu, biểu đồ theo tháng và nhóm món ăn.....33

Hình 3.25: Thẻ "Thông tin cá nhân" và khối "Hoạt động của tôi" 34

Hình 3.26: Thẻ "Sở thích ăn uống": món ưa thích, ngân sách và khoảng cách tối đa 34

Hình 3.27: Thẻ "Bảo mật tài khoản" – đổi mật khẩu có kiểm tra mật khẩu cũ 35

Hình 3.28: Thẻ "Tài khoản": ngày tạo, trạng thái, gói dịch vụ và xuất dữ liệu 36

Hình 3.29: Bảng điều khiển quản trị: hàng chờ kiểm duyệt và các chỉ số vận hành 37

Hình 3.30: Quản lý danh mục quán ăn và đề xuất của người dùng 37

Hình 3.31: Quản lý danh mục món ăn 38

Hình 3.32: Hàng chờ kiểm duyệt Food Note công khai 38

Hình 3.33: Nhật ký hoạt động của hệ thống 39

Hình 4.1: Bốn header bảo mật đặt cho mọi phản hồi API 40

Hình 4.2: Hàm chuẩn hoá đường dẫn yêu cầu về một cửa vào duy nhất.....40

Hình 4.3: Mở kết nối PDO: exception, prepared statement thật và múi giờ UTC....41

Hình 4.4: Trình tự xử lý một yêu cầu API có ghi dữ liệu (POST /food-notes) 42

Hình 4.5: Băm mật khẩu bằng Argon2id rồi chèn hàng users trong cùng một giao dịch 43

Hình 4.6: Hai lớp kiểm tra CSRF: token lưu trong phiên và Origin của yêu cầu.....44

Hình 4.7: Hàm rate_limit(): cửa sổ thời gian cố định và ngưỡng vượt 45

Hình 4.8: Trình tự xác minh ID token của Google Identity Services 46

Hình 4.9: Kiểm tra kid, chữ ký RS256 và toàn bộ claim bắt buộc của ID token 47

Hình 4.10: Sinh truy vấn Overpass QL theo tag, bán kính và giới hạn kết quả 49

Hình 4.11: Danh sách ba máy chủ Overpass và khởi tạo curl_multi.....50

Hình 4.12: Vòng polling nhận phản hồi đầu tiên hợp lệ rồi thoát bằng break 2.....50

Hình 4.13: Phục vụ tile đã đệm từ đĩa và danh sách máy chủ tile nguồn 52

Hình 4.14: GET_LOCK toàn cục và nhịp đệm 1,1 giây trước khi gọi Nominatim..53

Hình 4.15: Truy vấn tìm quán gần nhất với Haversine viết trực tiếp trong SQL 54

Hình 4.16: Hàm normalizeText(): slug hoá tiếng Việt để tìm kiếm không phân biệt dấu 55

Hình 4.17: Máy trạng thái của Food Note theo quyền riêng tư và trạng thái kiểm duyệt 56

Hình 4.18: Mười một đường dẫn được đưa vào Cache Storage khi service worker cài đặt 60

Hình 4.19: Hai điều kiện loại trừ tuyệt đối vùng động trong sự kiện fetch 60

Hình 4.20: Luồng khởi tạo và các công cụ vận hành của dự án 62

Hình 4.21: Sơ đồ triển khai và các thành phần chạy trên máy chủ.....63

DANH MỤC BẢNG BIỂU

Bảng 2.1: Danh mục công nghệ và vai trò trong dự án 6

Bảng 2.2: Bảng yêu cầu chức năng và nơi cài đặt 9

Bảng 2.3: Yêu cầu phi chức năng và cách đáp ứng trong mã nguồn.....10

Bảng 2.4: Chức năng của 23 bảng trong cơ sở dữ liệu 14

Bảng 4.1: Các bộ giới hạn tần suất được khai báo trong mã nguồn 45

Bảng 4.2: So sánh ba nhóm chủ thể theo các khả năng nghiệp vụ 48

Bảng 4.3: Các tầng bộ nhớ đệm và chính sách tuổi dữ liệu.....51

Bảng 4.4: Mức độ đầy đủ trường dữ liệu của snapshot 8.600 địa điểm 55

Bảng 5.1: Đối chiếu mục tiêu đề ra ở Chương 1 với kết quả kiểm chứng được trong mã nguồn.....66

DANH MỤC CÁC CHỮ VIẾT TẮT

Ký hiệu	Cụm từ đầy đủ
API	Application Programming Interface – giao thức lập trình ứng dụng
Argon2id	Thuật toán băm mật khẩu chống GPU, chuẩn PHC 2015
CSRF	Cross-Site Request Forgery – giả mạo yêu cầu từ site khác
CSS	Cascading Style Sheets – ngôn ngữ định dạng giao diện
EMU	English Metric Unit – đơn vị toạ độ trong OOXML
FK	Foreign Key – khoá ngoại
GeoJSON	Định dạng JSON mô tả đối tượng không gian
HTTP(S)	HyperText Transfer Protocol (Secure)
Haversine	Công thức tính khoảng cách trên mặt cầu
ID token	Token định danh trong chuẩn OpenID Connect
JWK / JWKS	JSON Web Key / tập khoá công khai của nhà cung cấp
JWT	JSON Web Token – token ký bằng chữ ký số
MSSV	Mã số sinh viên
PDO	PHP Data Objects – lớp truy cập cơ sở dữ liệu của PHP
PWA	Progressive Web App – ứng dụng web tiến bộ
REST	Representational State Transfer – kiểu kiến trúc API
RDBMS	Relational Database Management System
SEO	Search Engine Optimization
SLA	Service Level Agreement
SMTP	Simple Mail Transfer Protocol – giao thức gửi thư
SQL	Structured Query Language
SSR	Server-Side Rendering
TLS	Transport Layer Security
UI/UX	User Interface / User Experience
XSS	Cross-Site Scripting – chèn script độc hại

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Sinh viên ở gần ký túc xá và khu đô thị gặp hai vấn đề lặp lại mỗi ngày: không biết ăn gì, và không nhớ mình đã ăn ở đâu, hết bao nhiêu tiền.

Ứng dụng giao đồ ăn giải quyết phần đặt món nhưng không giúp khám phá quán quanh mình theo khoảng cách, giá và loại món. Sổ tay giấy thì không tra cứu, không thống kê được.

Về mặt học thuật, đây là bài toán tổng hợp nhiều nội dung đã học.

Gồm: phân tích yêu cầu, thiết kế cơ sở dữ liệu quan hệ, giao tiếp client – server qua API, tích hợp dịch vụ bản đồ của bên thứ ba, và xử lý an toàn thông tin cho mật khẩu, phiên làm việc, nội dung do người dùng tạo.

Nhóm chọn tự viết trên PHP thuần thay vì dùng một framework có sẵn.

Mục đích là kiểm soát trọn đường đi của một yêu cầu, từ rewrite của Apache tới câu lệnh SQL.

Cách này còn giúp mọi kỹ thuật trong báo cáo đều giải thích được bằng chính mã nguồn của dự án.

1.2 Mục tiêu của đề tài

1.2.1 Mục tiêu chung

Xây dựng một website hoàn chỉnh, chạy được thực tế, giúp người dùng khám phá quán ăn gần khu vực đang sống và ghi lại nhật ký ẩm thực cá nhân, đồng thời hỗ trợ quyết định ăn uống theo nhóm.

1.2.2 Mục tiêu cụ thể

- Khám phá: lọc quán theo danh mục, khoảng cách, mức giá và trạng thái đang mở; chọn ngẫu nhiên khi người dùng không biết ăn gì.
- Nhật ký ẩm thực (Food Note): ghi tên món, giá, điểm đánh giá, nhận xét, hình ảnh, nhãn; lưu nháp, chỉnh sửa và phân quyền hiển thị.

- Kế hoạch ăn nhóm (Food Plan): tạo kế hoạch, mời thành viên, mỗi người đề xuất quán và bình chọn, hệ thống xếp hạng theo số phiếu.
- Thống kê cá nhân: biểu đồ chi tiêu theo tháng, phân loại món, quán ghé nhiều nhất.
- Nền dữ liệu quán ăn: khai thác OpenStreetMap, có kiểm duyệt và bổ sung thông tin từ cộng đồng.
- An toàn: xác thực hai phương thức, phân quyền người dùng – quản trị, giới hạn tần suất, nhật ký hoạt động.
- Trải nghiệm: giao diện thích ứng ba cỡ màn hình, cài được như ứng dụng (PWA), xem offline nội dung đã tải.

1.3 Phạm vi và đối tượng sử dụng

Đối tượng phục vụ chính là người dùng trẻ, đặc biệt là sinh viên tại TP. Hồ Chí Minh.

Phạm vi dữ liệu bản đồ giới hạn ở khu vực thành phố này. Đó là quyết định có chủ đích để kiểm soát khối lượng truy vấn OSM và chất lượng dữ liệu, không phải hệ thống phủ toàn quốc.

Về chức năng, đề tài làm cả bốn phần: khám phá, ghi chép cá nhân, kế hoạch nhóm và quản trị nội dung. Đề tài không làm: đặt bàn, thanh toán online, giao hàng, định giá chính xác của quán, và ứng dụng di động native.

1.4 Phương pháp thực hiện

Dự án đi theo vòng lặp nhỏ: phân tích mô tả đề tài, thiết kế bảng, viết API, làm giao diện, kiểm thử bằng trình duyệt và bằng script, rồi rà soát lại mã nguồn.

Toàn bộ quyết định kỹ thuật được ghi trong tài liệu nội bộ của repo (README.md,database_design.md,COMPLETION20261028.md,DEPLOYMENT.md, tiendo.md) để đối chiếu khi viết báo cáo.

Một nguyên tắc làm việc được áp dụng xuyên suốt: không ghi nhận dữ liệu là "đã xác minh" khi chưa có bằng chứng.

Vì vậy mã nguồn phân biệt rõ verified_active, reported_closed và unverified, thay vì mặc định coi toàn bộ dữ liệu OSM là quán đang mở.

1.5 Bố cục báo cáo

Báo cáo gồm năm chương.

Chương 1 nêu bối cảnh và mục tiêu. Chương 2 phân tích cơ sở lý thuyết, kiến trúc, yêu cầu và thiết kế cơ sở dữ liệu. Chương 3 mô tả giao diện và quy trình sử dụng. Chương 4 đi sâu vào kỹ thuật triển khai và kiểm thử. Chương 5 kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH HỆ THỐNG

Chương này trả lời ba câu hỏi: hệ thống được dựng bằng công nghệ gì và vì sao, cấu trúc mã nguồn ra sao, và dữ liệu được tổ chức thế nào. Mọi nhận định đều chiếu theo mã nguồn trong repo what2eat.

2.1 Công nghệ sử dụng

2.1.1 PHP 8 theo mô hình API không framework

PHP là ngôn ngữ kịch phí phía máy chủ: mã thực thi khi có yêu cầu HTTP và trả văn bản về trình duyệt.

What2Eat không dùng Laravel, Symfony hay Slim mà tự viết một lớp định tuyến gọn trong api/index.php.

Hàm handle_request() nối tiếp các nhánh kiểm tra đường dẫn và phương thức; mỗi nhánh gọi thẳng PDO rồi trả JSON qua json_response().

Cách làm này có hai lợi thế với một đồ án: đường đi của một yêu cầu đọc được từ trên xuống dưới trong một file, không phải lần theo tầng trung gian; và không phụ thuộc vòng đời phiên bản của bên thứ ba.

Đổi lại, những thứ framework thường lo sẵn – kiểm thực dữ liệu, chống CSRF, giới hạn tần suất, ghi nhật ký – nhóm đều phải tự viết và tự chịu trách nhiệm bảo mật.

api/bootstrap.php · dòng 102–109 · đường ra JSON chuẩn của toàn hệ thống

```
function json_response(array $data, int $status = 200): never
{
    http_response_code($status);
    header('Content-Type: application/json; charset=utf-8');
    header('Cache-Control: no-store, private');
    echo json_encode($data, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES |
JSON_INVALID_UTF8_SUBSTITUTE);
    exit;
}
```

Hình 2.1: Hàm json_response(): đường ra JSON chuẩn của toàn hệ thống (Nguồn: api/bootstrap.php:102-109)

Ba cờ json_encode ở trên là chi tiết đáng chú ý: JSON_UNESCAPED_UNICODE giữ nguyên dấu tiếng Việt thay vì escapes thành \u1ea1, JSON_INVALID_UTF8_SUBSTITUTE tránh toàn bộ response bị fail chỉ vì một byte không hợp lệ trong dữ liệu do người dùng nhập.

2.1.2 MySQL 8 và PDO

Cơ sở dữ liệu dùng InnoDB với bảng mã utf8mb4_unicode_ci, lưu được tiếng Việt đầy đủ và hỗ trợ giao dịch.

Kiểu JSON của MySQL được dùng ở chín cột cho những dữ liệu có hình dạng thay đổi: sở thích người dùng, giờ mở cửa, payload địa điểm OSM, danh sách hiệu chỉnh, nhãn ghi chú.

Truy cập dữ liệu đi qua PDO với bốn thiết lập được khai báo ngay khi mở kết nối; đoạn mã dưới chép nguyên văn từ tệp.

```
// api/bootstrap.php:69-74 – nguyên văn khối khởi tạo PDO
        $pdo = new PDO($dsn, $user, $password, [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
            PDO::MYSQL_ATTR_INIT_COMMAND => "SET time_zone = '+00:00'",
        ]);
```

Về an toàn, dòng quan trọng nhất là ATTR_EMULATE_PREPARES = false: PHP gửi câu lệnh sang MySQL chuẩn bị thật, nên giá trị truyền vào chỉ là tham số, không bao giờ bị ghép chuỗi vào cú pháp SQL.

Múi giờ phiên đặt về +00:00 và mọi câu ghi dùng UTC_TIMESTAMP(), nhờ đó dữ liệu thời gian nhất quán dù máy chủ đổi giờ.

2.1.3 Frontend: HTML, CSS, JavaScript thuần, Bootstrap, Leaflet, Chart.js

Giao diện gồm 20 trang PHP: 1 trang chủ, 13 trang người dùng trong pages/, 6 trang quản trị trong admin/. Mỗi trang là một tài liệu HTML độc lập kèm CSS riêng.

Bootstrap 5.3 nạp từ CDN cho hệ thống lưới và vài component. Phần lớn giao diện viết bằng CSS tự đặt tên lớp w2e-*, không phụ thuộc hình dáng mặc định của framework.

Toàn bộ dữ liệu trên trang được tải bằng fetch() sau khi trang hiện ra, tức mô hình SPA-lite trên nền nhiều trang: máy chủ chỉ gửi khung HTML, trình duyệt gọi API và tự dựng danh sách. Leaflet 1.9.4 vẽ bản đồ; Chart.js 4 vẽ biểu đồ ở Dashboard.

2.1.4 OpenStreetMap, Overpass API và Nominatim

OpenStreetMap (OSM) là bản đồ do cộng đồng đóng góp, dữ liệu mở theo giấy phép ODbL.

What2Eat dùng ba dịch vụ của hệ OSM: Overpass API truy vấn các đối tượng là quán ăn trong một bán kính, Nominatim chuyển địa chỉ thành toạ độ và ngược lại, máy chủ tile vẽ nền bản đồ.

Lý do chọn OSM thay vì Google Maps: không cần khoá thương mại, không phát sinh chi phí theo yêu cầu, và được lưu lại dữ liệu quán do người dùng bổ sung.

Hạn chế là chất lượng dữ liệu không đồng đều, nên hệ thống phải có lớp kiểm chứng và dự phòng, trình bày ở Chương 4.

2.1.5 Google Identity Services

Đăng nhập bằng Google theo luồng ID token: trình duyệt nhận token đã ký từ Google, gửi về máy chủ, máy chủ xác minh chữ ký rồi mới tin.

Dự án không dùng thư viện kiểm tra token. Toàn bộ việc giải mã base64url, dựng khoá công khai từ JWK sang chuẩn DER/SPKI và gọi openssl_verify được viết tay trong api/google-auth.php (158 dòng).

2.1.6 Progressive Web App

PWA là web app cài được lên thiết bị và chạy khi mất mạng, gồm ba mảnh: manifest.json khai báo tên, màu, biểu tượng; service worker sw.js chặn yêu cầu để phục vụ từ bộ nhớ đệm; offline.html làm màn hình dự phòng.

What2Eat đặt tên cache là what2eat-shell-v7-district-import và chỉ đệm 11 đường dẫn tĩnh; toàn bộ API và dữ liệu động bị loại trừ.

2.1.7 Tổng hợp công nghệ

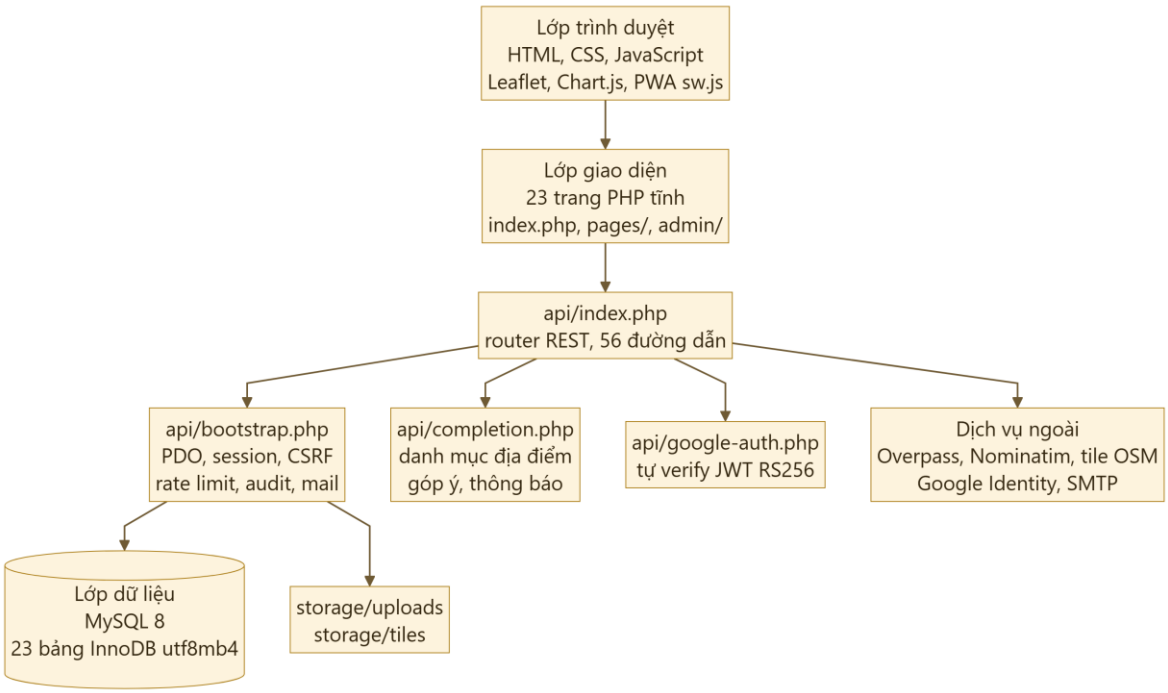
Bảng 2.1: Danh mục công nghệ và vai trò trong dự án

Công nghệ	**Vai trò**	**Lý do chọn / hạn chế**
PHP 8.1+ (không framework)	REST API, render trang	Kiểm soát trọn đường đi request; phải tự viết bảo mật
MySQL 8 / MariaDB 10.4	Lưu toàn bộ dữ liệu nghiệp vụ	Giao dịch, khoá ngoại, kiểu JSON; một số hàm mới không có trên MariaDB
PDO	Lớp truy cập CSDL	Prepared statement thật, chống SQL injection
HTML/CSS + Bootstrap 5.3	Khung giao diện, responsive	Lưới sẵn có; CSS tự viết chiếm phần lớn
JavaScript thuần (5 file, ~3.000 dòng)	Tải dữ liệu, dựng DOM, tương tác	Không build step, đọc được trực tiếp; tự quản lý state
Leaflet 1.9.4	Bản đồ OSM	Nhẹ, mã mở, dùng được tile OSM
Chart.js 4	Biểu đồ Dashboard	Dòng, tròn, cột; cấu hình ngắn
Overpass API	Truy vấn quán ăn theo toạ độ	Miễn phí nhưng chậm và hay quá tải → phải đua 3 endpoint
Nominatim	Geocoding 2 chiều	Miễn phí, giới hạn 1 yêu cầu/giây → dùng GET_LOCK
Google Identity Services	Đăng nhập Google	Giảm rào cản đăng ký; phải tự kiểm tra JWT
PHPMailer 7 (Composer)	Gửi mail xác minh, đặt lại mật khẩu	SMTP có TLS; cần composer install
Service Worker + manifest	PWA, offline	Chỉ cache phần shell, dữ liệu động không cache

Node.js (dev only)	Chạy kiểm thử tích hợp	Không phải thành phần vận hành

2.2 Kiến trúc hệ thống

2.2.1 Ba lớp và một cửa vào



Hình 2.2: Kiến trúc tổng thể của What2Eat

(Nguồn: Mã nguồn dự án what2eat)

Hệ thống có bốn lớp.

Lớp trình bày là các tệp HTML, JS, CSS chạy trong trình duyệt. Lớp giao diện là 20 trang PHP tĩnh, chỉ xuất khung HTML và nạp asset, không truy vấn CSDL. Lớp ứng dụng là api/ với một cửa vào duy nhất. Lớp dữ liệu gồm MySQL, storage/uploads cho ảnh và storage/cache cho tile bản đồ.

Mọi yêu cầu dữ liệu đi qua cùng một đường.

Tệp .htaccess ở gốc có khối mod_rewrite với bảy quy tắc, trong đó quy tắc ^api(?:/.*)?$ gom mọi đường dẫn con của API về api/index.php.

Tệp api/.htaccess không rewrite mà chỉ chặn truy cập trực tiếp vào các tệp PHP khác trong thư mục api/, nên index.php là tệp duy nhất chạy được.

Nhờ hai tệp này, /api/v1/food-notes/12 không cần tồn tại như một file thật.

.htaccess · dòng 14–24 · định tuyến Apache đưa mọi đường dẫn API về api/index.php

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteCond %{HTTP_HOST} ^what2eat[.]local(?::80)?$ [NC]
  RewriteRule ^ http://what2eat.localhost%{REQUEST_URI} [R=302,L,NE]
  RewriteRule (^|/)[.] - [F,L]
  RewriteRule ^api/index[.]php$ - [L]
  RewriteRule ^api(?:/.*)?$ api/index.php [QSA,L]
  RewriteRule ^tiles/([0-9]{1,2})/([0-9]{1,8})/([0-9]{1,8})[.]png$ api/index.php?
tile=$1/$2/$3 [QSA,L]
  RewriteRule ^offline[.]html$ - [L]
  RewriteRule ^(.+)[.]html$ /$1.php [R=302,L,NE]
</IfModule>
```

Hình 2.3: Bảy quy tắc rewrite trong tệp .htaccess ở gốc dự án

(Nguồn: .htaccess:14-24)

Hai quy tắc đáng chú ý nhất là dòng 21 và dòng 23.

Dòng 21 biến ảnh bản đồ /tiles/17/56/108.png thành một endpoint động có đệm đĩa.

Dòng 23 cho phép thân URL dùng đuôi .html trong khi file thật là .php, nên đường dẫn trông tĩnh và không lộ tên công nghệ máy chủ.

Dòng 18 chặn mọi nỗ lực đọc tệp ẩn bằng cách trả 403 ngay ở Apache.

2.2.2 Phân tách trách nhiệm trong api/

File	Dòng	Trách nhiệm
api/index.php	1.312	Định tuyến toàn bộ endpoint, logic OSM, tile, geocode, xác thực, kiểm thử duyệt
api/bootstrap.php	317	Kết nối PDO, session, hàm kiểm thực dữ liệu, CSRF, giới hạn tần suất, audit, gửi mail
api/completion.php	131	Danh mục địa điểm place_catalog, hiệu chỉnh cộng đồng, thông báo, tìm kiếm gộp ba nguồn
api/google-auth.php	158	Tự kiểm tra ID token Google: base64url, JWK sang DER/SPKI, openssl_verify, cache JWKS
api/bin/*.php	5 file	Công cụ dòng lệnh: migrate, create-admin, cleanup, import-catalog, enrich-addresses

Cách chia file theo hướng "một cửa vào, nhiều module ngang" khiến đồ thị phụ thuộc rất đơn giản: index.php require bootstrap.php rồi gọi tiếp completion.php và google-auth.php; không có module nào gọi ngược lại index.php.

2.2.3 Bề mặt API

Phân tích toàn bộ điều kiện trong api/index.php và api/completion.php cho thấy hệ thống có 56 đường dẫn, tương ứng 76 cặp đường dẫn – phương thức.

Số đường dẫn là đáng tin vì mỗi đường dẫn chỉ xuất hiện một lần. Nhóm /food-notes/{id} dùng một khối if duy nhất rồi rẽ nhánh theo phương thức bên trong, nên số cặp phương thức là xấp xỉ.

Nhóm	Số lượng	Ví dụ đại diện
GET	30	/health, /restaurants, /food-plans/{id}, /dashboard, /admin/audit-logs
POST	23	/auth/register, /auth/google, /food-notes, /favorites, /place-corrections
PATCH	8	/food-notes/{id}, /food-plans/{id}, /admin/place-reports/{id}
PUT	5	/profile, /admin/restaurants/{id}, /admin/restaurants/{id}/sponsorship
DELETE	10	/food-notes/{id}, /food-plans/{id}, /profile, /admin/users/{id}

2.3 Phân tích yêu cầu

2.3.1 Yêu cầu chức năng

Bảng 2.2: Bảng yêu cầu chức năng và nơi cài đặt

Mã	Nội dung	API chính	Mã UI
TC01	Đăng ký, kích hoạt email, đăng nhập, đặt lại mật khẩu	/auth/register, /auth/verify, /auth/login, /auth/reset	pages/register.php, login.php, reset-password.php
TC02	Đăng nhập bằng tài khoản Google	/auth/google	assets/js/google-auth.js

TC03	Khám phá quán ăn theo bản đồ, bộ lọc, khoảng cách	/restaurants, /restaurants/search	pages/discovery.php, restaurant-map.js
TC04	Xem chi tiết quán, chỉ đường, lưu yêu thích	/restaurants/{id}, /favorites	pages/restaurant.php
TC05	Ghi, sửa, xoá nhật ký ẩm thực kèm ảnh và nhãn	/food-notes, /food-notes/{id}, /food-notes/{id}/media	pages/food-notes.php, create-food-note.php
TC06	Phân quyền hiển thị ghi chú: riêng tư / công khai / qua liên kết	/food-notes/{id}/share	site.js
TC07	Lên kế hoạch nhóm, mời, đề xuất, bình chọn	/food-plans, /food-plans/{id}/invite, /proposals, /vote	pages/food-plans.php
TC08	Thống kê cá nhân và biểu đồ	/dashboard	pages/dashboard.php
TC09	Góp ý, báo sai thông tin địa điểm	/place-corrections, /place-reports	restaurant-map.js popup
TC10	Quản trị: quán ăn, danh mục, ghi chú, người dùng, báo cáo	/admin/*	admin/*.php
TC11	Nhật ký hoạt động và xác minh trạng thái địa điểm	/admin/audit-logs, /admin/osm-verifications	admin/audit-logs.php
TC12	Thông báo trong ứng dụng	/notifications, /notifications/read	site.js
TC13	Cài đặt ứng dụng và xem offline	—	sw.js, manifest.json, offline.html

2.3.2 Yêu cầu phi chức năng

Bảng 2.3: Yêu cầu phi chức năng và cách đáp ứng trong mã nguồn

Nhóm	**Yêu cầu**	**Cách đáp ứng**
Bảo mật	Không lưu mật khẩu dạng rõ	password_hash() thuật toán Argon2id, tự nâng cấp khi đăng nhập

Bảo mật	Chống giả mạo yêu cầu	CSRF token 32 byte, so sánh hash_equals, kiểm tra thêm Origin
Bảo mật	Chống đoán dò	Bảng rate_limits tính theo cửa sổ thời gian và HMAC của IP
Bảo mật	Thu hồi phiên khi đổi mật khẩu	Cột session_version, so sánh ở mọi request
Bảo mật	Không lộ file cấu hình	.htaccess chặn .env, *.sql, *.log, README, storage/, includes/
Hiệu năng	Không spam Overpass	Đệm api_cache 10 phút, stale 24 giờ, snapshot tĩnh
Hiệu năng	Giảm tải tile server	Proxy /tiles với đệm đĩa 7 ngày (604.800 giây)
Hiệu năng	Truy vấn theo toạ độ nhanh	Index ix_restaurants_coords và tiền lọc bằng hộp bao
Khả dụng	OSM lỗi vẫn dùng được	Ba tầng dự phòng, ưu tiên dữ liệu cache cũ hơn là trả lỗi
Dễ dùng	Giao diện ba cỡ màn hình	Bootstrap grid + CSS tự viết, mobile-nav.js
Dễ truy cập	Ngôn ngữ rõ, phản hồi bằng văn bản	Thuộc tính aria-*, role, live region, nút có nhãn
Kiểm thử	Có bộ kiểm tra tự động	tests/integration.mjs tạo CSDL tạm, chạy 12 nhóm chức năng

2.3.3 Phân quyền

Hệ thống có ba vai: khách (không đăng nhập), người dùng và quản trị viên.

Khách được xem bản đồ, tìm quán, đọc ghi chú công khai; không được ghi chú, lưu quán, tham gia kế hoạch hay báo lỗi.

Người dùng làm các thao tác cá nhân và chỉ thấy dữ liệu của mình cùng nội dung công khai.

Quản trị viên duyệt ghi chú, quản lý quán, danh mục, người dùng, xử lý báo cáo và xác minh trạng thái địa điểm.

Kiểm tra quyền làm hoàn toàn phía máy chủ.

current_user() đọc session, đối chiếu session_version và chỉ lấy tài khoản có status = active. require_admin() gọi tiếp current_user() rồi chặn nếu role khác admin.

Trang HTML không chặn – ai cũng tải được admin/index.php – nhưng mọi dữ liệu và thao tác đều bị API từ chối. Đây là mô hình bảo mật theo lớp chứ không theo giao diện.

2.4 Thiết kế cơ sở dữ liệu

2.4.1 Quy ước

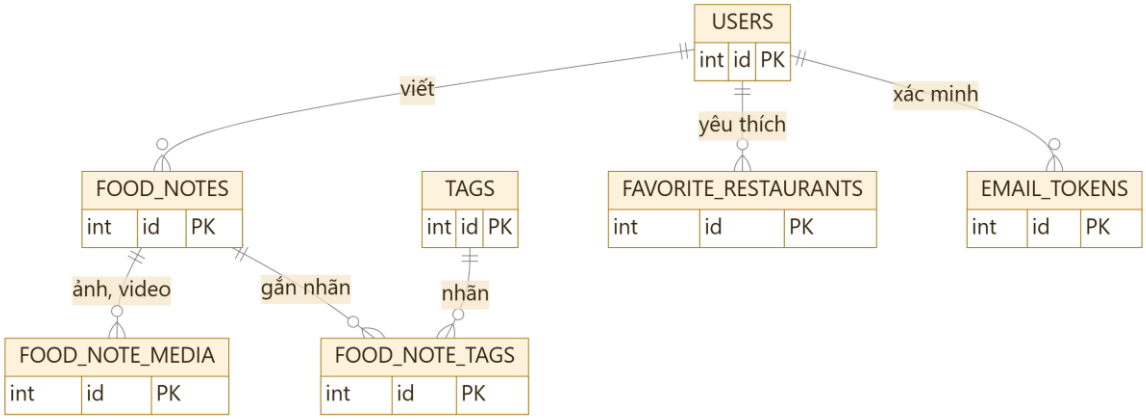
Toàn bộ bảng nằm trong một cơ sở dữ liệu quan hệ, đặt tên theo số ít hoặc số nhiều tự nhiên, khoá chính là id kiểu INT AUTO_INCREMENT, trừ bốn bảng liên kết – trạng thái dùng khoá tự nhiên: favorite_restaurants(user_id, source, source_id), votes(proposal_id, user_id), rate_limits(bucket, subject_hash, window_start), osm_place_blocks(source_id).

Múi giờ lưu trữ là UTC ở mọi cột thời gian; câu lệnh dùng UTC_TIMESTAMP() và UTC_DATE() thay vì NOW() và CURDATE(). Trạng thái dùng ENUM để ràng buộc ngay tại CSDL thay vì kiểm tra bằng code.

2.4.2 Sơ đồ thực thể – quan hệ

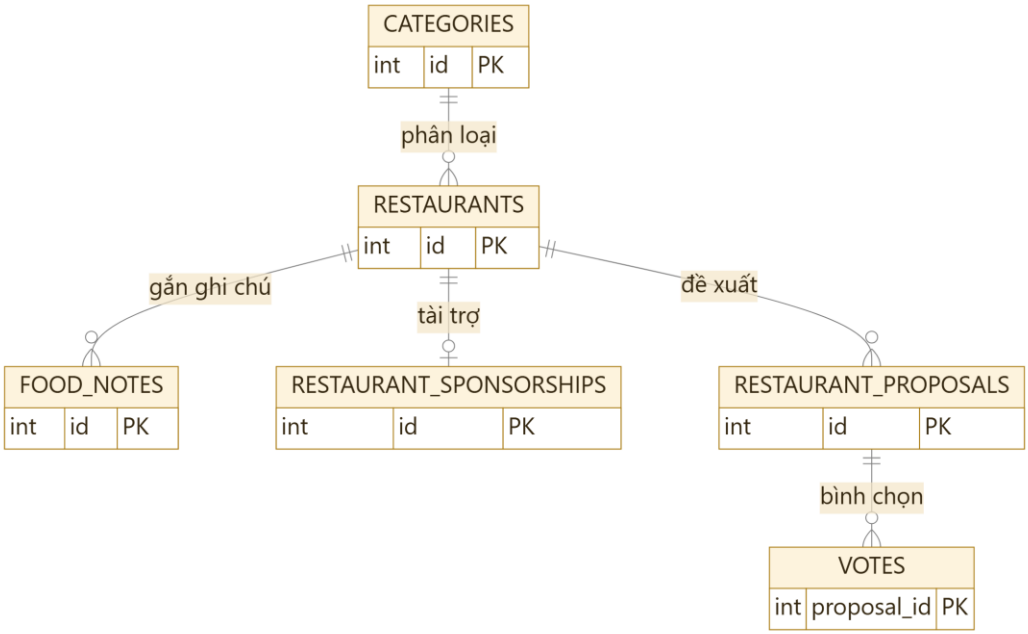
Hai mươi ba bảng chia thành bốn nhóm chức năng.

Nhóm người dùng và ghi chú chứa dữ liệu có vòng đời gắn với tài khoản. Nhóm quán ăn kết nối danh mục, đề xuất, tài trợ. Nhóm kế hoạch ăn quản lý quan hệ mời – đề xuất – bình chọn. Nhóm dữ liệu địa điểm và vận hành giữ bản sao OSM, hiệu chỉnh cộng đồng, báo cáo, đệm API và nhật ký.



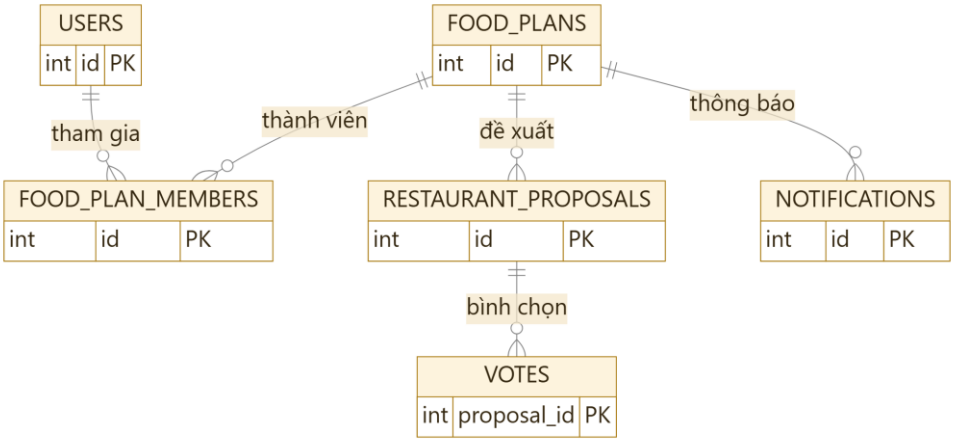
Hình 2.4: ERD nhóm người dùng, nhật ký ẩm thực, nhãn và media

(Nguồn: api/schema.sql)



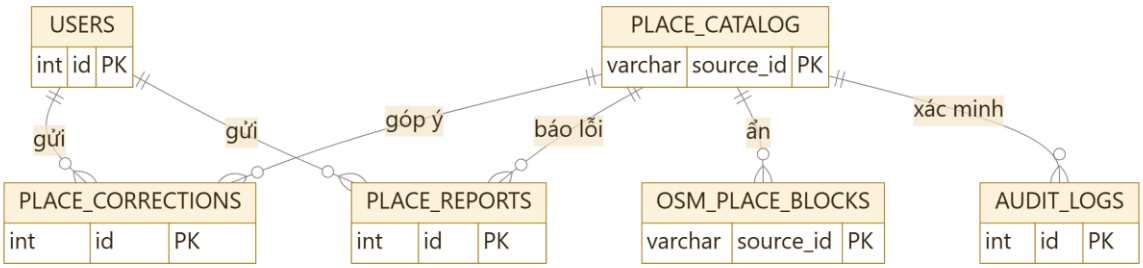
Hình 2.5: ERD nhóm danh mục, quán ăn, tài trợ và đề xuất

(Nguồn: api/schema.sql)



Hình 2.6: ERD nhóm kế hoạch ăn và bình chọn

(Nguồn: api/schema.sql)



Hình 2.7: ERD nhóm dữ liệu địa điểm OSM và vận hành

(Nguồn: api/schema.sql)

2.4.3 Danh mục bảng

Bảng 2.4: Chức năng của 23 bảng trong cơ sở dữ liệu

Bảng	**Dòng điển hình**	**Chức năng**
users	1 người = 1 dòng	Tài khoản, mật khẩu đã băm, google_sub, vai trò, sở thích JSON, session_version
email_tokens	Ngắn hạn	Token kích hoạt và đặt lại mật khẩu, lưu dạng băm, có hạn dùng
categories	~10	Danh mục món ăn, slug duy nhất, trạng thái active/hidden
restaurants	Quản trị nhập	Quán do hệ thống quản lý, toạ độ, giờ mở cửa JSON, trạng thái duyệt

restaurant_sponsorships	Theo hợp đồng	Tài trợ có kỳ hạn, CHECK ends_at > starts_at
food_notes	Trung tâm	Nhật ký ẩm thực: món, giá, điểm, privacy, status, share_token_hash
tags / food_note_tags	Nhiều-nhiều	Nhãn do người dùng tự tạo và bảng nối
food_note_media	1 ảnh = 1 dòng	storage_key, mime_type, loại ảnh/video, quyền xem
favorite_restaurants	Khoá kép	Quán yêu thích, trỏ được tới cả quán OSM lẫn quán trong CSDL
food_plans	1 bữa nhóm	Kế hoạch, trạng thái open/decided/closed/cancelled, invite_token_hash
food_plan_members	Thành viên	Quan hệ người – kế hoạch, vai trò owner/member
restaurant_proposals	1 đề xuất	Quán được giới thiệu trong kế hoạch, UNIQUE theo kế hoạch + nguồn
votes	1 phiếu	Khoá kép (proposal_id, user_id) nên mỗi người chỉ một phiếu cho một đề xuất
rate_limits	Tự xoá định kỳ	Đếm thao tác theo bucket, HMAC của IP và cửa sổ thời gian
api_cache	Tự xoá định kỳ	Kết quả Overpass và Nominatim dạng JSON kèm hạn dùng
audit_logs	Tăng liên tục	Nhật ký hành động, ip_hash dạng HMAC, metadata JSON
place_reports	Cộng đồng gửi	Báo quán đóng cửa / sai thông tin, hàng chờ cho quản trị
osm_place_blocks	Quản trị ẩn	Danh sách địa điểm OSM bị ẩn khỏi bản đồ
place_catalog	8.600	Danh mục địa điểm OSM đã nhập, payload JSON, trạng thái xác minh
place_corrections	Cộng đồng gửi	Hiệu chỉnh địa chỉ, điện thoại, giờ mở cửa, có duyệt
notifications	Trong ứng dụng	Thông báo cho thành viên kế hoạch ăn
food_plan_members (FK)	—	Ràng buộc CASCADE khi xoá người dùng hoặc kế hoạch

schema_migrations	8	Đánh dấu phiên bản cấu trúc đã áp dụng

2.4.4 Ràng buộc toàn vẹn

Ngoài khoá chính, schema khai báo 18 khoá ngoại ON DELETE CASCADE, 13 khoá ON DELETE SET NULL, 3 ràng buộc CHECK, 19 index thường và 9 UNIQUE KEY.

Ba ràng buộc CHECK gồm: điểm đánh giá trong khoảng 0 tới 5, giá tiền không âm, kỳ hạn tài trợ phải kết thúc sau khi bắt đầu.

Việc chọn CASCADE hay SET NULL là quyết định nghiệp vụ.

Ghi chú, phiếu bầu, thành viên kế hoạch xoá theo người dùng vì chúng vô nghĩa khi chủ tài khoản không còn.

Ngược lại, audit_logs.actor_id, place_corrections.user_id và food_notes được SET NULL để giữ dấu vết kiểm duyệt và nội dung do cộng đồng đóng góp.

api/schema.sql · dòng 130–135 · index, UNIQUE KEY và CHECK của bảng food_notes

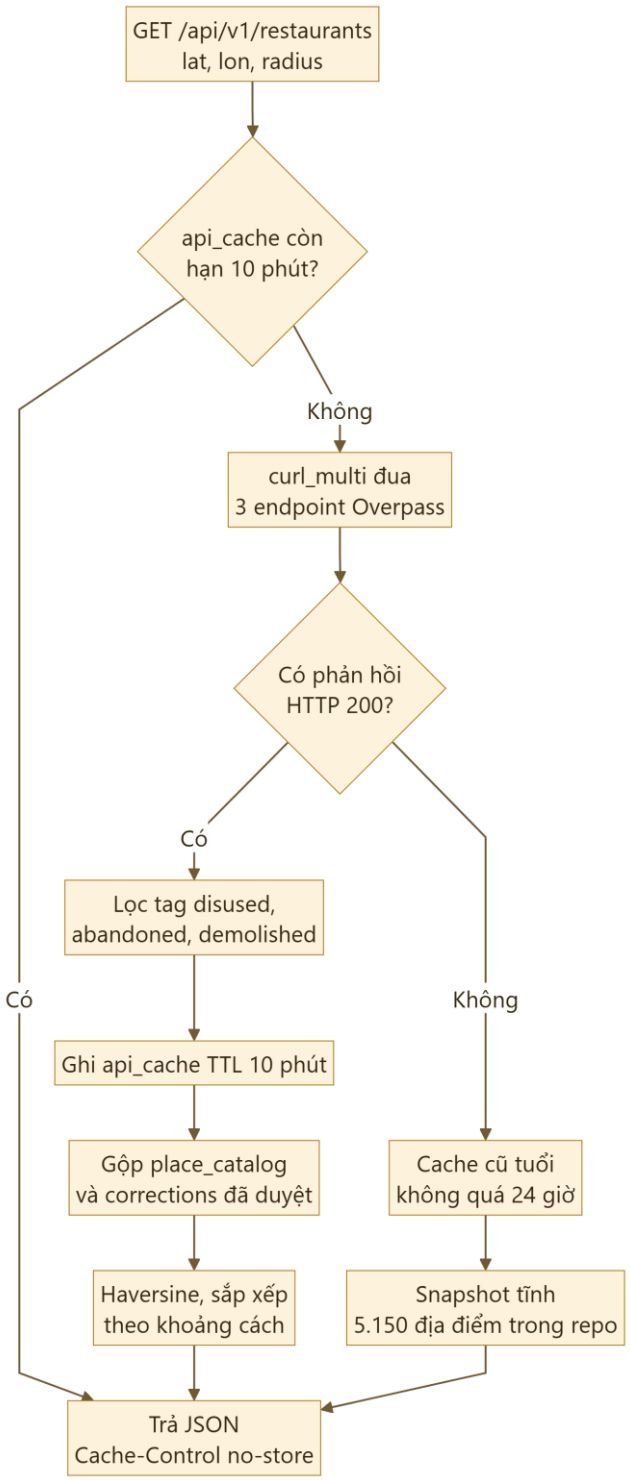
```
  UNIQUE KEY uq_food_notes_share_token (share_token_hash),
  CONSTRAINT fk_food_notes_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE
CASCADE,
  CONSTRAINT fk_food_notes_restaurant FOREIGN KEY (restaurant_id) REFERENCES
restaurants(id) ON DELETE SET NULL,
  CONSTRAINT fk_food_notes_moderator FOREIGN KEY (moderated_by) REFERENCES users(id) ON
DELETE SET NULL,
  CONSTRAINT ck_food_notes_rating CHECK (rating IS NULL OR (rating >= 0 AND rating <=
5)),
  CONSTRAINT ck food notes price CHECK (price IS NULL OR price >= 0)
```

Hình 2.8: Khoá, index và ràng buộc CHECK của bảng food_notes

(Nguồn: api/schema.sql:130-135)

Ràng buộc ở tầng CSDL là lớp cuối cùng: kể cả khi một nhánh code nào đó bỏ qua kiểm tra phía PHP, MySQL vẫn từ chối ghi dữ liệu sai.

2.5 Luồng dữ liệu địa điểm OpenStreetMap



Hình 2.9: Luồng xử lý và các tầng dự phòng khi lấy dữ liệu quán ăn

(Nguồn: api/index.php:520-578)

Sơ đồ trên là đường đi của request /api/v1/restaurants.

Điểm đáng chú ý là chiến lược chịu lỗi: hệ thống thà trả dữ liệu cũ còn hơn trả lỗi.

Nếu Overpass không trả kết quả, máy chủ lấy bản ghi đệm đã quá hạn nhưng còn trong 24 giờ; nếu cũng không có thì dùng snapshot tĩnh 8.600 địa điểm TP. Hồ Chí Minh đưa thẳng vào repo.

Sau khi có dữ liệu thô, kết quả được làm giàu bằng place_catalog – nơi lưu những gì cộng đồng và quản trị viên đã bổ sung – rồi sắp xếp theo khoảng cách tính bằng công thức Haversine. Chi tiết kỹ thuật của từng tầng nằm ở Chương 4.

2.6 Đánh giá lựa chọn kiến trúc

Kiến trúc một cửa vào phù hợp với quy mô đồ án: toàn bộ quy tắc nghiệp vụ nằm trong bốn file PHP, dễ lần theo lỗi, dễ kiểm thử.

Bù lại, api/index.php đã hơn 1.300 dòng và sẽ khó bảo trì nếu chức năng tiếp tục tăng. Hướng tách route thành nhiều file nhỏ được nêu ở Chương 5.

Tương tự, không dùng framework giúp nhóm hiểu rõ từng lớp bảo mật mình tạo ra, nhưng cũng không có bản vá tự động cho các lỗ hổng thông thường.

Đây là sự đánh đổi có ý thức, phù hợp mục tiêu học thuật của đề tài.

CHƯƠNG 3. GIAO DIỆN VÀ QUY TRÌNH SỬ DỤNG

Chương này trình bày toàn bộ giao diện của What2Eat theo đúng thứ tự một người dùng mới đi qua: trang chủ, tạo tài khoản, khám phá quán ăn bằng bản đồ, ghi nhật ký ẩm thực, lên kế hoạch nhóm, xem thống kê cá nhân, khu vực quản trị.

Mỗi màn hình kèm một ảnh chụp và mô tả ngắn về chức năng, dữ liệu hiển thị và luồng thao tác.

Toàn bộ 33 ảnh trong chương là ảnh chụp từ hệ thống đã triển khai, không phải mô hình thiết kế.

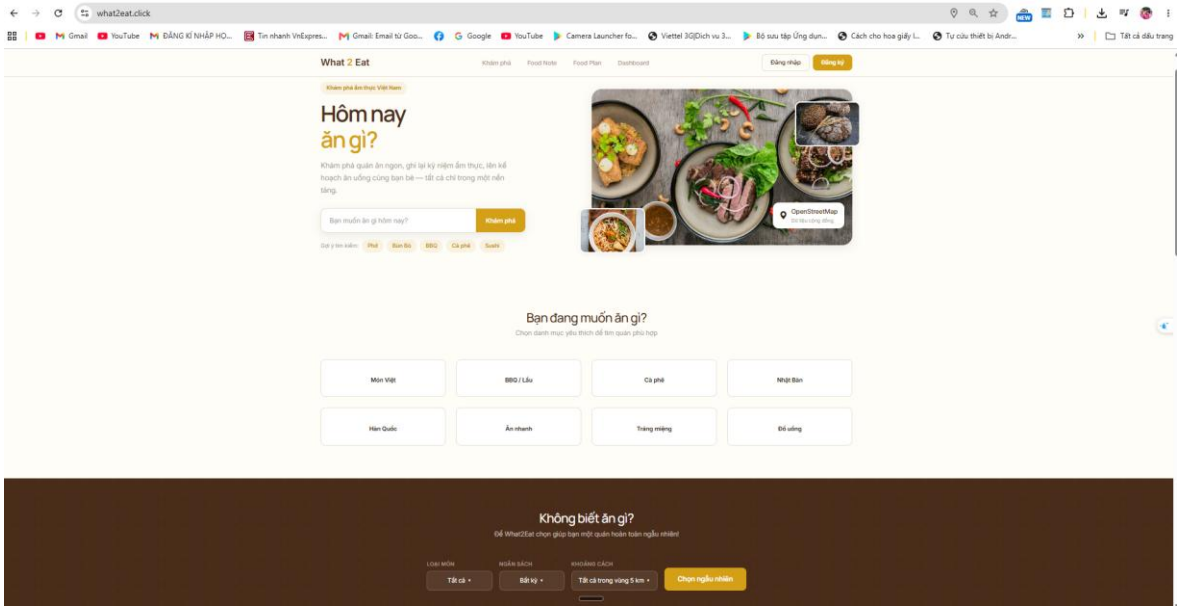
Nhãn giao diện, tên nút và tuỳ chọn bộ lọc được giữ nguyên như trong mã nguồn trang (thư mục pages/ và admin/) để người đọc đối chiếu trực tiếp.

3.1 Trang chủ và phiên khách

Trang chủ (index.php) là nơi người dùng chưa đăng nhập vẫn xem được nội dung.

Trang mở đầu bằng khối định vị "Hôm nay – Bạn đang muốn ăn gì?" và ô "Không biết ăn gì?" cho phép chọn ngẫu nhiên.

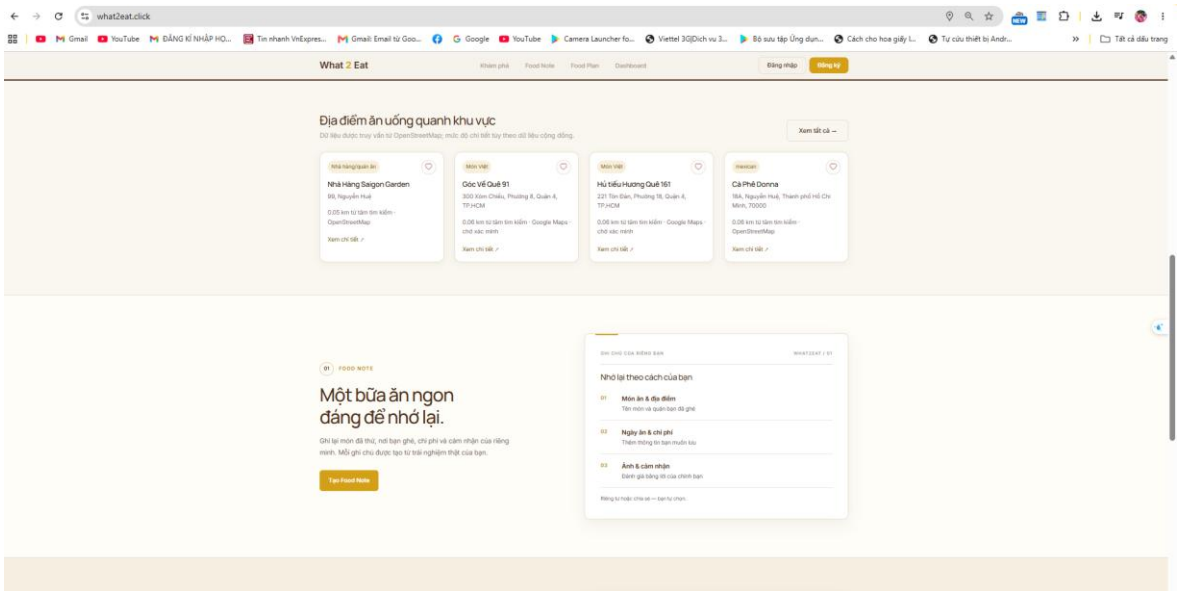
Tiếp đó là danh sách "Địa điểm ăn uống quanh khu vực", khối giới thiệu "Một bữa ăn ngon" và lời mời "Cả nhóm cùng chọn, Hôm nay ăn gì?".



Hình 3.1: Đầu trang chủ: định vị thương hiệu và các gợi ý nhanh

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

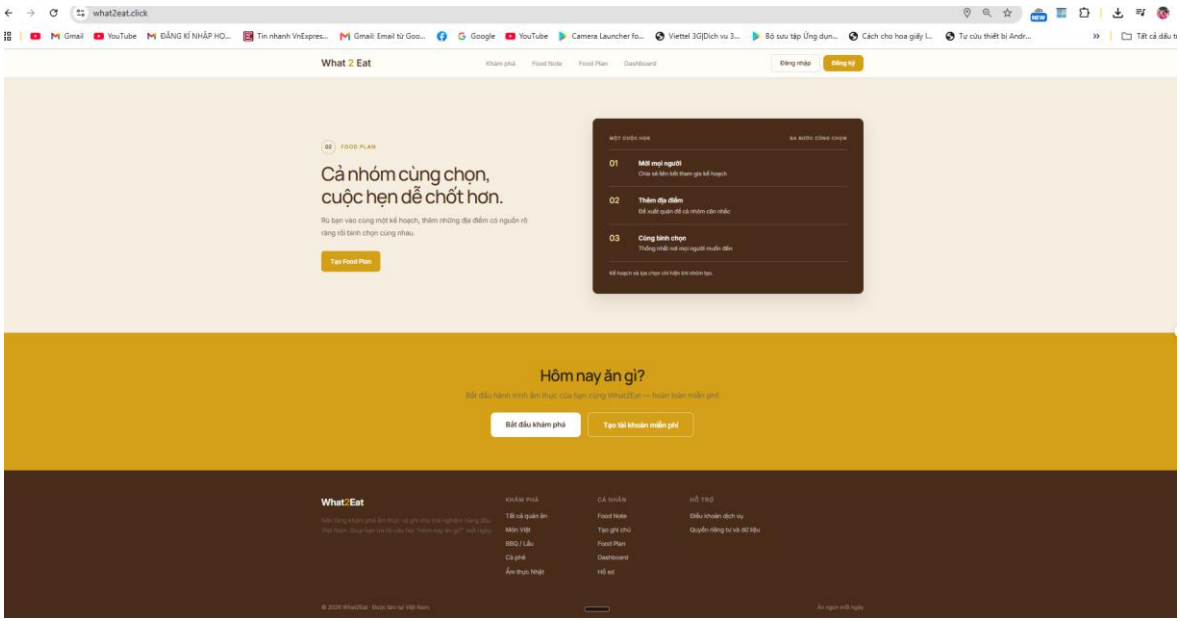
Phần đầu trang chứa thanh điều hướng cố định gồm Trang chủ, Khám phá, Food Notes, Food Plans, Dashboard và menu tài khoản. Khi chưa đăng nhập, menu này hiển thị "Khách" cùng hai liên kết Đăng nhập và Đăng ký.



Hình 3.2: Giữa trang chủ: danh sách địa điểm nổi bật và khối giới thiệu tính năng

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Danh sách địa điểm được lấy qua API công khai, ưu tiên bản tin tài trợ đang hiệu lực rồi mới tới các quán đã duyệt. Mỗi thẻ hiển thị tên, danh mục, khoảng cách và trạng thái đang mở, tính từ toạ độ lưu trong cơ sở dữ liệu.



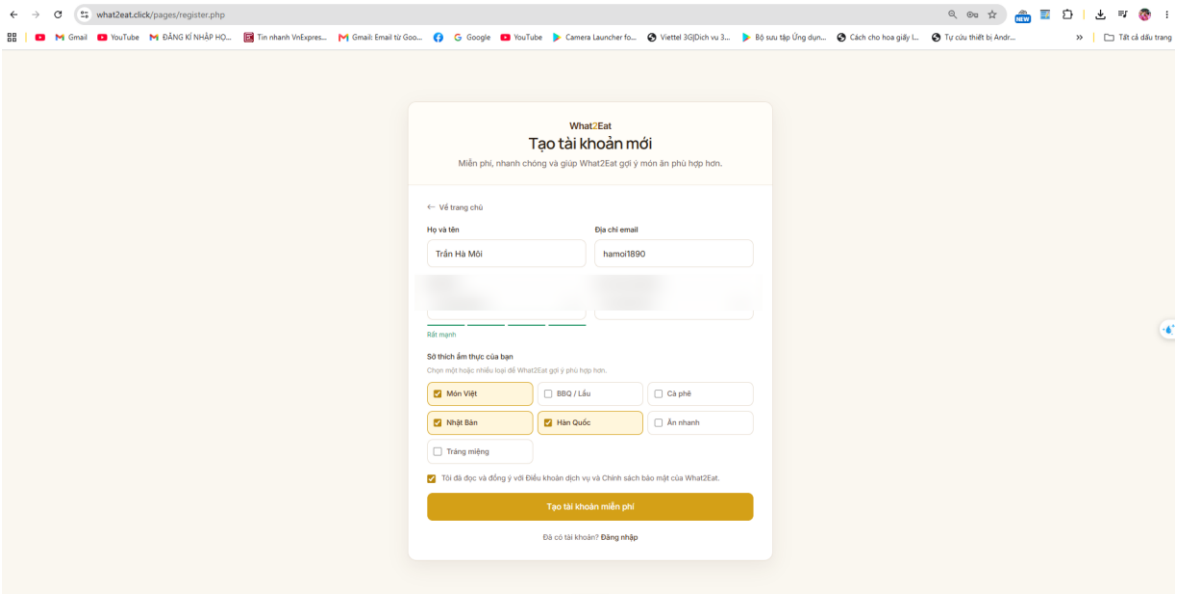
Hình 3.3: Cuối trang chủ: khu vực Food Plan nhóm và footer

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Ở cuối trang, lời mời tạo kế hoạch ăn nhóm dẫn thẳng tới trang Food Plan. Footer ghi rõ nguồn dữ liệu bản đồ theo yêu cầu ghi công của OpenStreetMap.

3.2 Đăng ký, đăng nhập và xác minh email

Luồng vào hệ thống gồm ba màn hình: tạo tài khoản, đăng nhập bằng mật khẩu và đăng nhập bằng Google. Sau khi đăng ký, tài khoản ở trạng thái pending và chưa thể đăng nhập cho tới khi người dùng bấm liên kết xác minh gửi qua email.



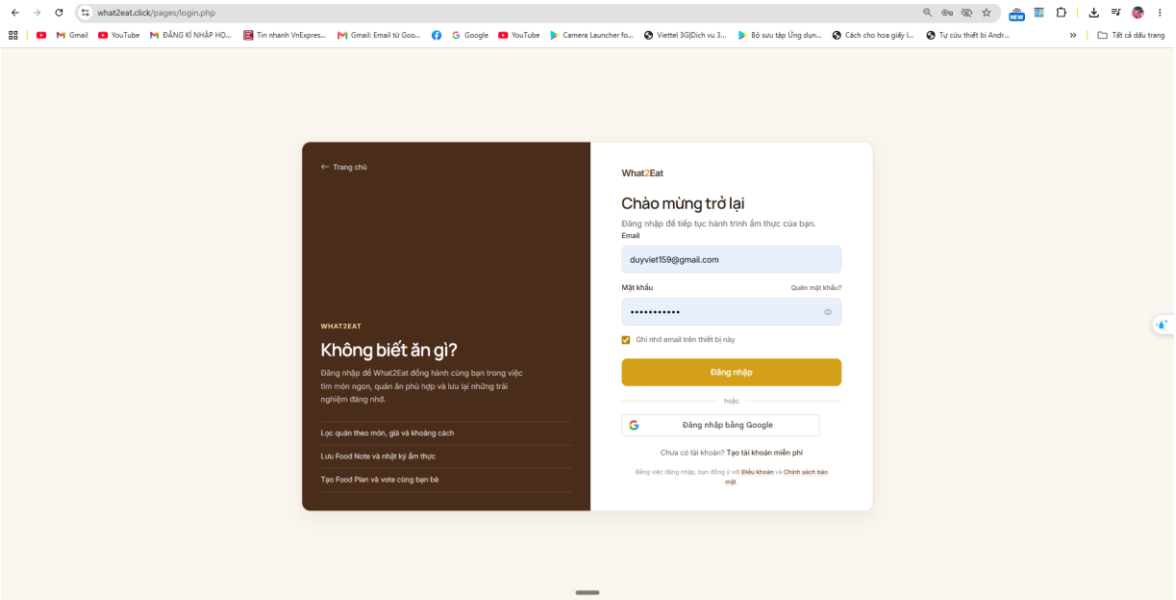
Hình 3.4: Màn hình "Tạo tài khoản mới" với biểu thức chính quy kiểm độ mạnh mật khẩu

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Biểu mẫu yêu cầu họ tên, email và mật khẩu tối thiểu 8 ký tự, có chữ hoa, chữ thường, chữ số và ký tự đặc biệt.

Mật khẩu được băm bằng Argon2id ngay khi nhận yêu cầu; hệ thống không lưu mật khẩu gốc ở bất kỳ đâu.

Nội dung hai ô mật khẩu trong ảnh đã được làm mờ.

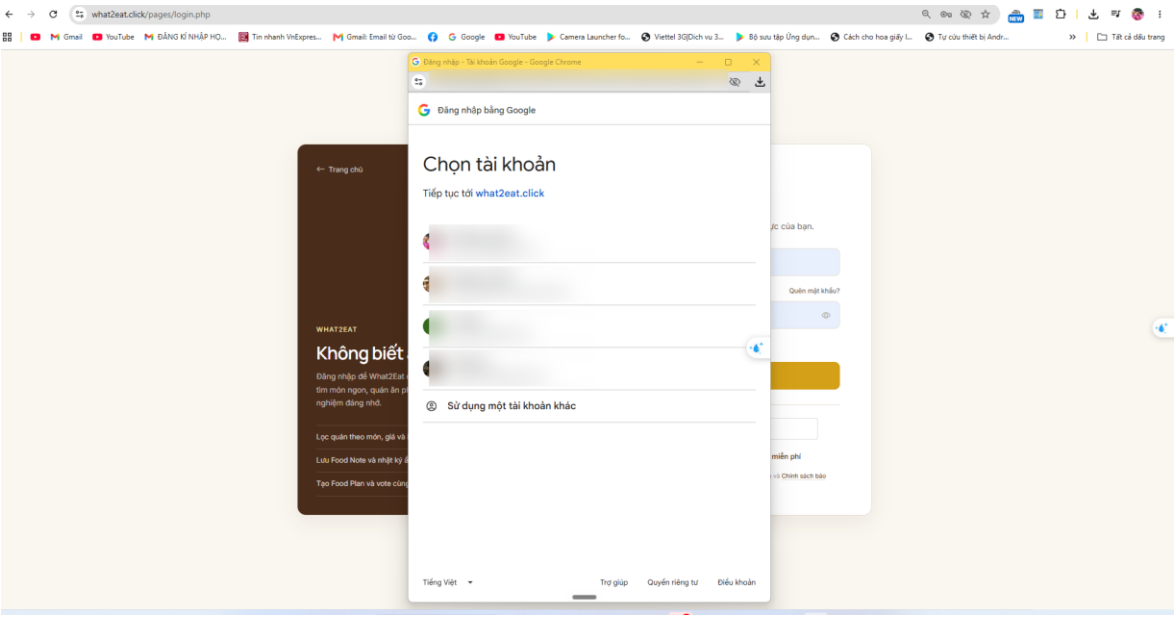


Hình 3.5: Màn hình "Chào mừng trở lại" – đăng nhập bằng email và mật khẩu

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Trang đăng nhập cũng là nơi người dùng quên mật khẩu: liên kết "Quên mật khẩu" sinh một token một lần có thời hạn và gửi qua SMTP.

Nếu nhập sai nhiều lần, máy chủ chặn bằng giới hạn tần suất 10 lần mỗi 15 phút theo địa chỉ IP đã băm.



Hình 3.6: Nút "Tiếp tục với Google" và hộp đồng ý của Google Identity Services

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Nút Google dùng luồng ID token: trình duyệt nhận token đã ký từ Google rồi gửi về máy chủ; máy chủ tự xác minh chữ ký RS256 trước khi chấp nhận danh tính.

Nếu email Google đã tồn tại trong hệ thống, tài khoản được liên kết một lần duy nhất qua cột google_sub.

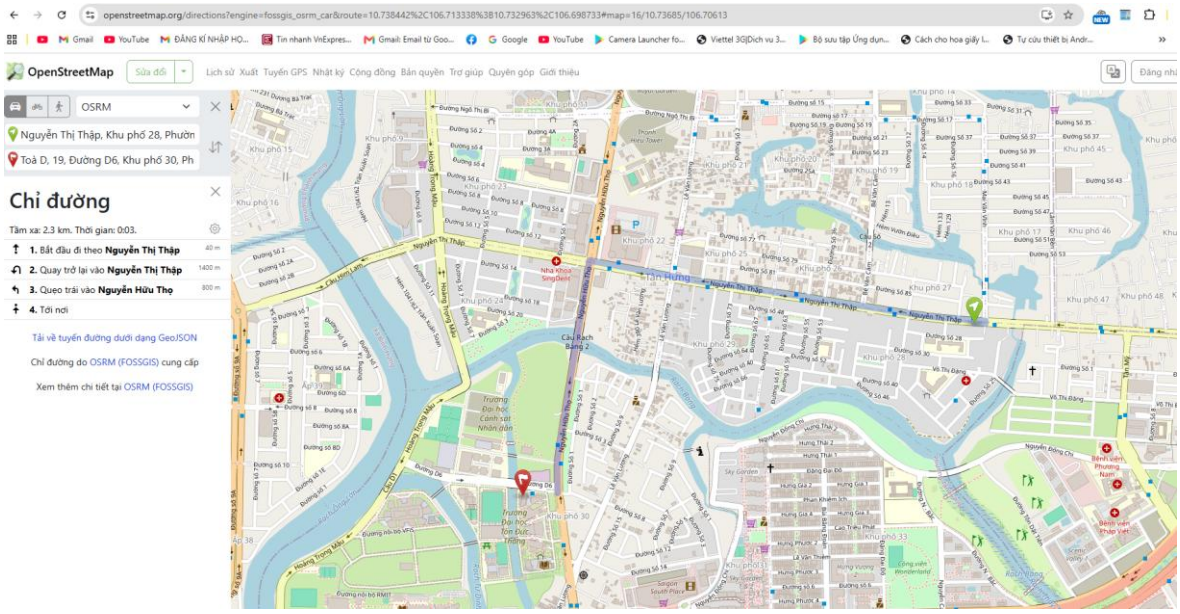
Danh sách tài khoản Google và liên kết xác thực trong ảnh đã được làm mờ vì chứa email của nhiều tài khoản.

3.3 Khám phá quán ăn theo vị trí

Trang Khám phá (pages/discovery.php) là chức năng trung tâm của hệ thống.

Giao diện gồm bản đồ Leaflet chiếm phần lớn màn hình, thanh bộ lọc phía trên và danh sách kết quả bên cạnh.

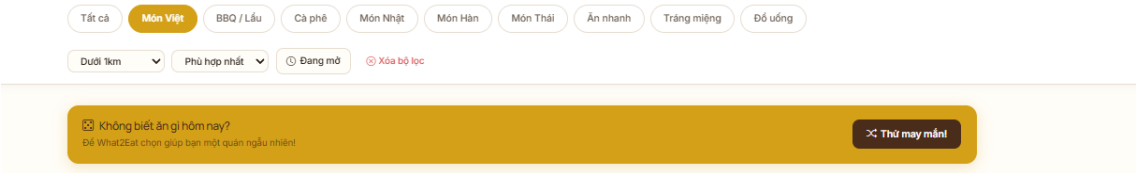
Bốn hộp thoại phụ: chọn vị trí, xem chi tiết, bổ sung thông tin, báo tin sai.



Hình 3.7: Bản đồ khám phá với các marker địa điểm và lời chú dẫn OpenStreetMap

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Marker được vẽ từ kết quả Overpass API trong bán kính người dùng chọn. Ảnh nền bản đồ không tải trực tiếp từ máy chủ OpenStreetMap mà đi qua proxy /tiles/ của dự án để có thể đệm trên đĩa (xem Chương 4).

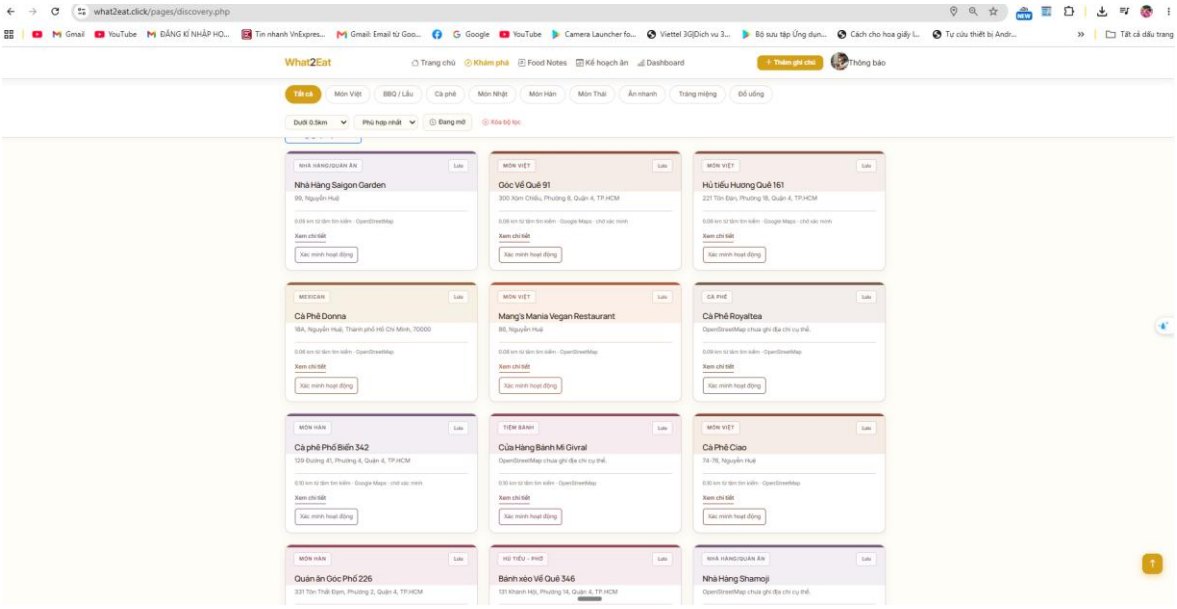


Hình 3.8: Thanh bộ lọc: Khoảng cách, Mức giá, Đánh giá và thứ tự sắp xếp

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Bộ lọc khoảng cách có năm mức (Dưới 0.5km, 1km, 2km, 5km, 10km); mức giá có Tất cả, Dưới 100k, Trên 200k; sắp xếp theo Phù hợp nhất, Gần nhất, Giá thấp hoặc Đánh giá cao.

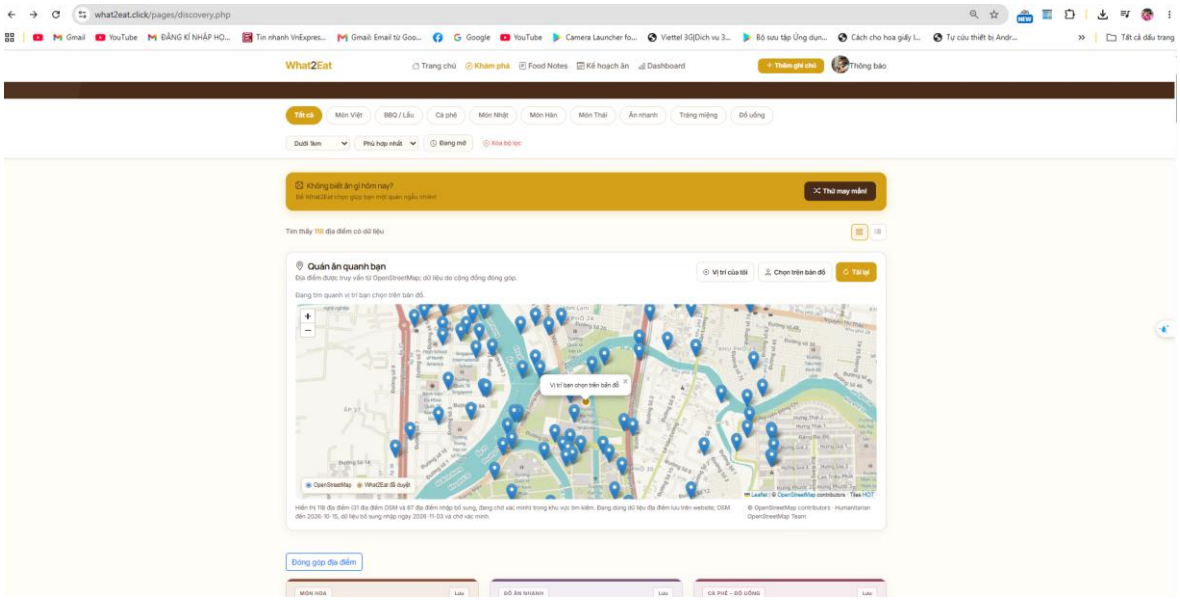
Toàn bộ lọc và sắp xếp chạy ở trình duyệt trên tập dữ liệu đã tải về, nên đổi bộ lọc không phát sinh yêu cầu mạng mới.



Hình 3.9: Danh sách địa điểm quét được trong khu vực đang xem

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

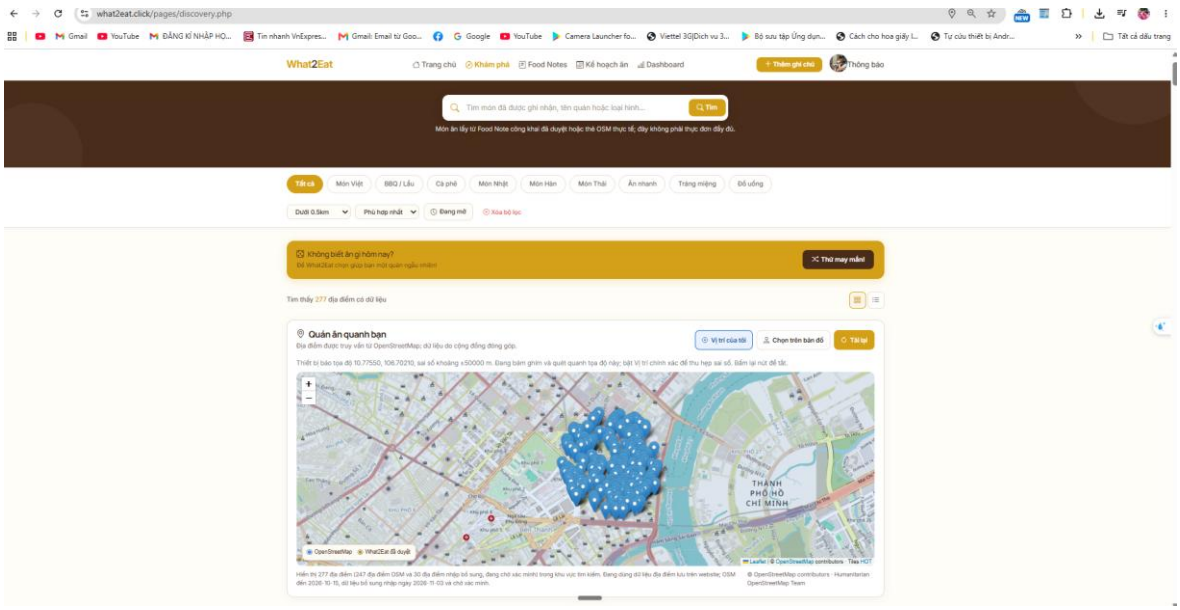
Danh sách hiển thị tên, món ăn đặc trưng, giá và trạng thái mở cửa. Khi dữ liệu lớn, trình duyệt chỉ dựng DOM cho một tập hữu hạn rồi bổ sung dần khi người dùng cuộn, tránh nghẽn trang.



Hình 3.10: Hộp thoại "Chọn địa điểm ngẫu nhiên" khi người dùng không biết ăn gì

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

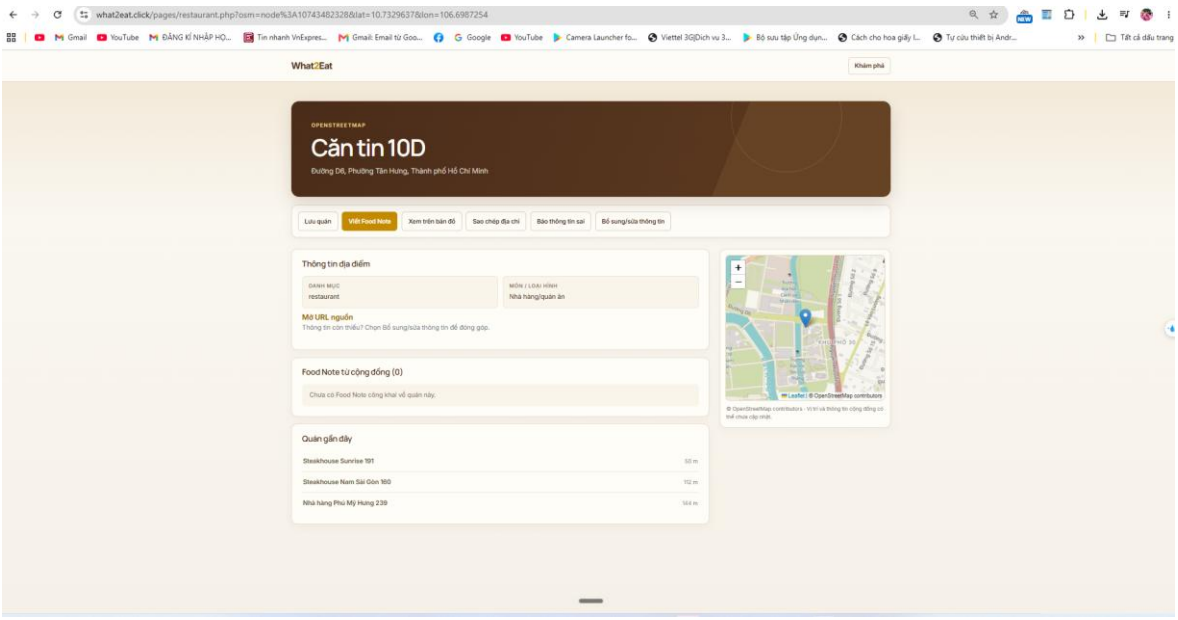
Nút này lấy mẫu ngẫu nhiên trong danh sách đã lọc thay vì trong toàn bộ cơ sở dữ liệu, nhờ vậy kết quả vẫn tôn trọng khoảng cách và ngân sách người dùng đang chọn.



Hình 3.11: Xác định vị trí hiện tại bằng Geolocation API của trình duyệt

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

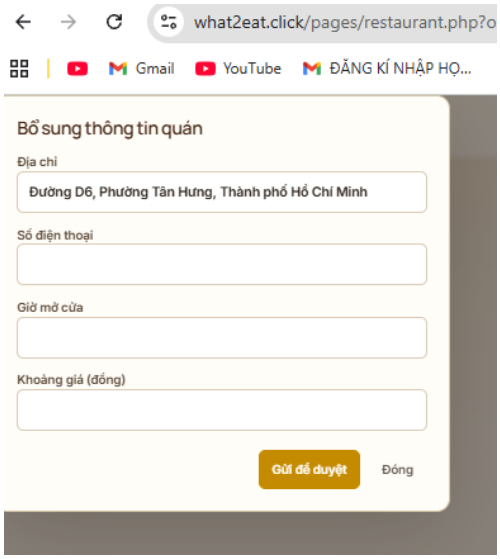
Vị trí lấy từ trình duyệt, không lưu lên máy chủ. Header Permissions-Policy trong .htaccess chỉ cho phép geolocation với nguồn cùng gốc, từ chối bên thứ ba.



Hình 3.12: Chi tiết một địa điểm: thông tin, đánh giá, nhật ký ẩm thực liên quan

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

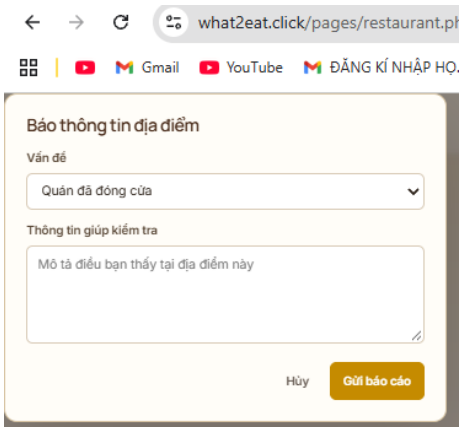
Trang chi tiết gộp ba nguồn: dữ liệu địa điểm, danh sách món ăn công khai từng được người dùng ghi (truy vấn kết hợp privacy='public' AND status='published') và các tuỳ chọn thao tác như thêm vào yêu thích hoặc mở Food Note cho quán này.



Hình 3.13: Biểu mẫu "Bổ sung thông tin" cho một địa điểm

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Người dùng có thể gửi số điện thoại, giờ mở cửa, mức giá hoặc website còn thiếu. Đề xuất không ghi thẳng vào dữ liệu gốc mà rơi vào bảng place_reports với trạng thái pending, chờ quản trị viên duyệt.



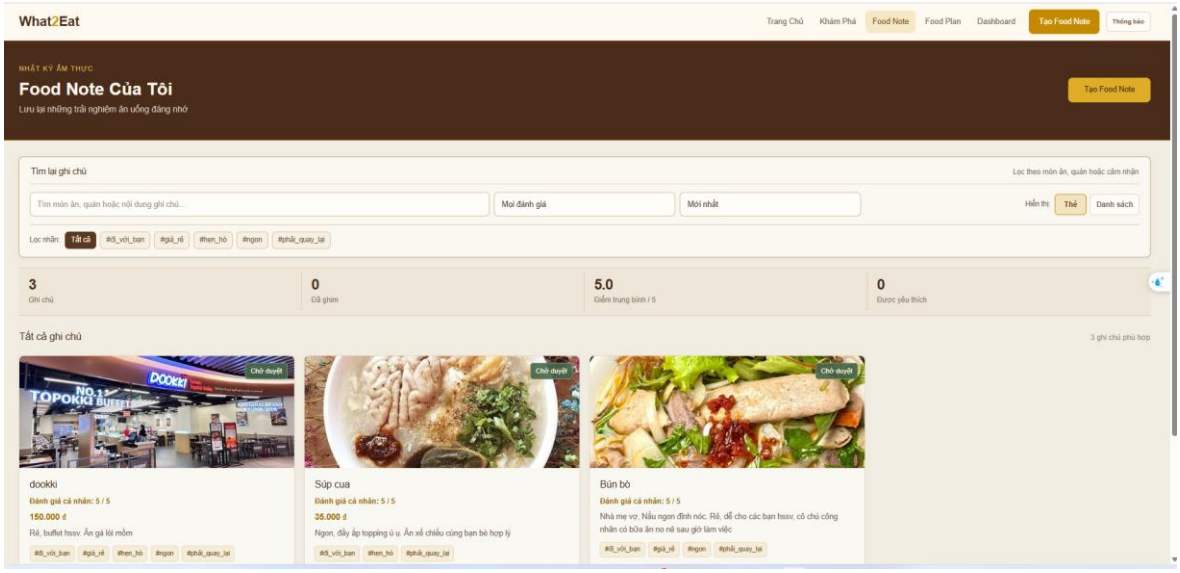
Hình 3.14: Biểu mẫu "Báo thông tin sai"

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Mỗi người dùng chỉ có một báo cáo pending cho cùng một địa điểm; máy chủ kiểm tra điều này trước khi ghi để tránh spam. Địa chỉ bị báo sai nhiều lần có thể bị chặn khỏi danh mục qua bảng osm_place_blocks.

3.4 Nhật ký ẩm thực cá nhân

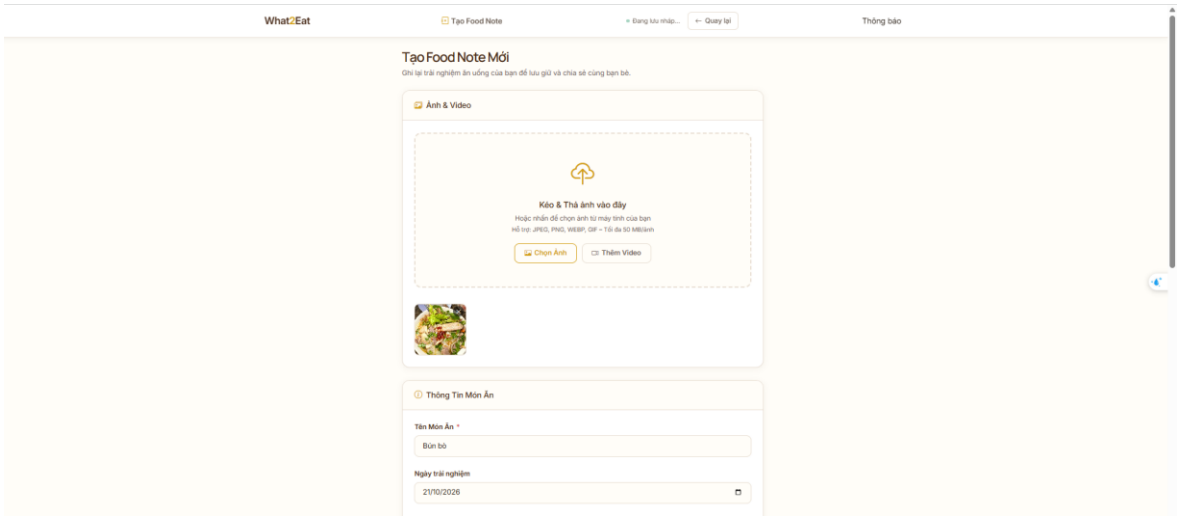
Food Note là đơn vị ghi chép của hệ thống: một món ăn, tại một quán, vào một ngày, kèm giá, điểm đánh giá, nhận xét, nhãn và hình ảnh. Người dùng quyết định quyền riêng tư cho từng ghi chú: private, link hoặc public.



Hình 3.15: Trang "Food Note Của Tôi" với bộ đếm tổng, ghim, yêu thích và điểm trung bình

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

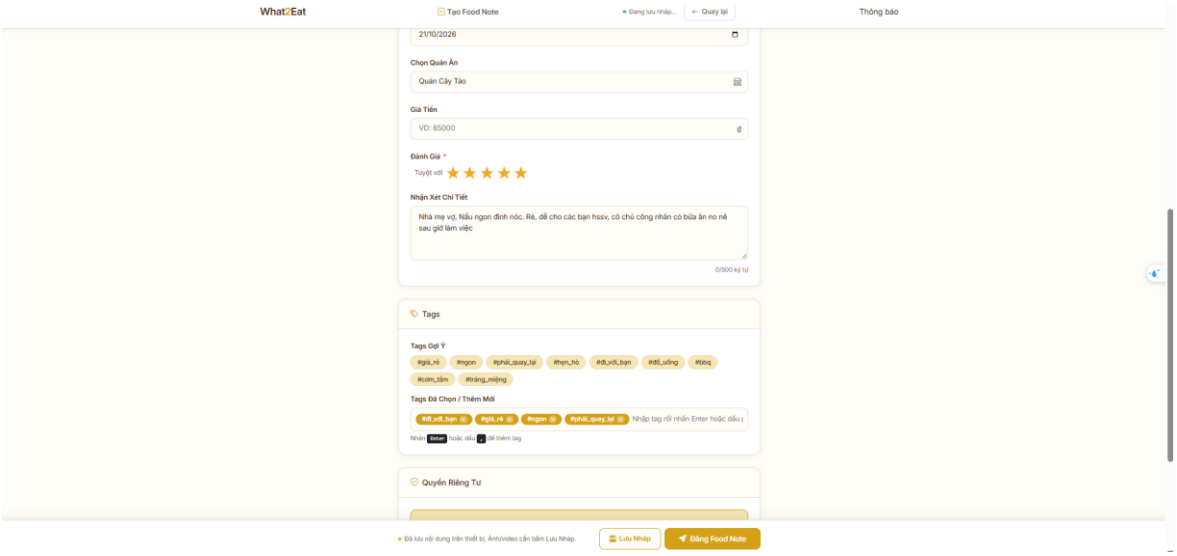
Đầu trang là bảng tổng hợp do máy chủ tính bằng một truy vấn đếm duy nhất; phía dưới là lưới thẻ ghi chú kèm thanh tìm "Tìm lại ghi chú". Người dùng chuyển được giữa phạm vi riêng tư và phạm vi cộng đồng.



Hình 3.16: Bước 1 của biểu mẫu tạo ghi chú: tên món, ngày ăn, giá và đánh giá

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

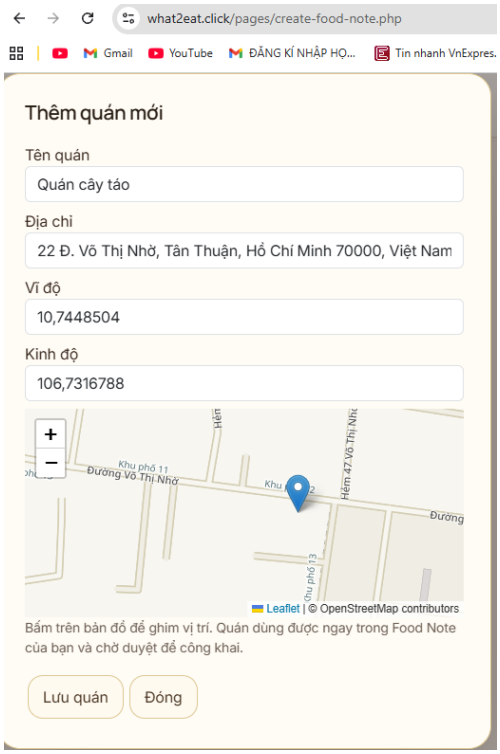
Biểu mẫu dùng một trường ẩn is_draft để phân biệt "Lưu nháp" và "Công khai". Nếu chọn công khai, máy chủ chuyển bản ghi sang trạng thái pending để quản trị viên duyệt thay vì hiển thị ngay.



Hình 3.17: Bước 2: nhận xét, nhãn và tải ảnh minh hoạ cho món ăn

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Ảnh được gửi bằng multipart/form-data, kiểm tra MIME thật bằng finfo rồi mới lưu vào storage/uploads với tên ngẫu nhiên 48 ký tự hex. Ghi chú có thể gắn nhiều nhãn; nhãn mới được tạo tự động và ghi rõ người tạo.



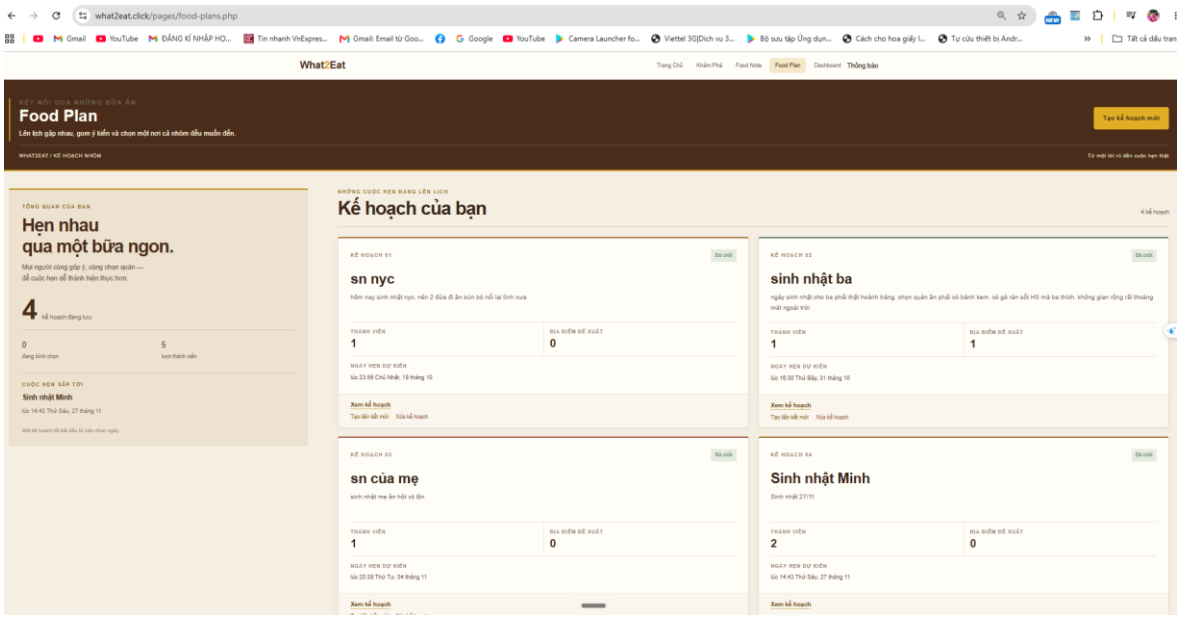
Hình 3.18: Thêm địa điểm mới ngay trong biểu mẫu ghi chú

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Khi quán chưa có trong danh mục, người dùng nhập tên và địa chỉ; hệ thống tạo bản ghi restaurants với status='pending'. Nhờ snapshot 8.600 địa điểm TP. Hồ Chí Minh, trường hợp này hiếm khi xảy ra.

3.5 Kế hoạch ăn uống theo nhóm

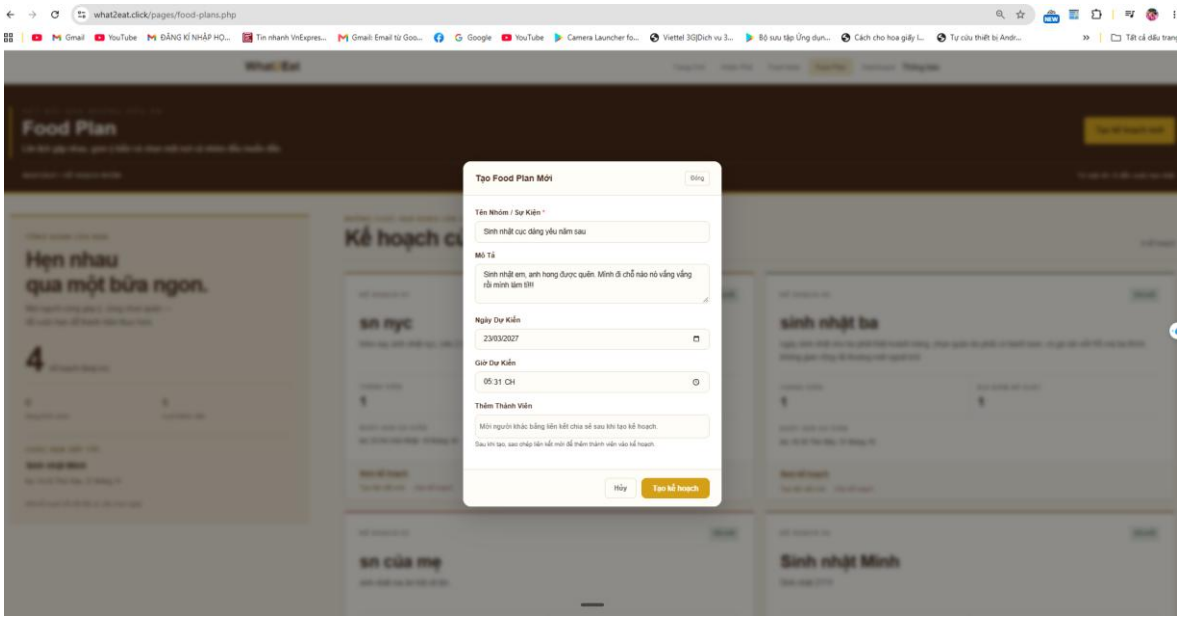
Food Plan giải quyết tình huống "đi mấy người mà không quán nào chịu đi". Chủ kế hoạch tạo nhóm, gửi liên kết mời, các thành viên đề xuất quán và bình chọn; máy chủ tổng hợp số phiếu và xếp hạng.



Hình 3.19: Danh sách "Kế hoạch của bạn" – cả kế hoạch đang mở lẫn đã quyết định

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

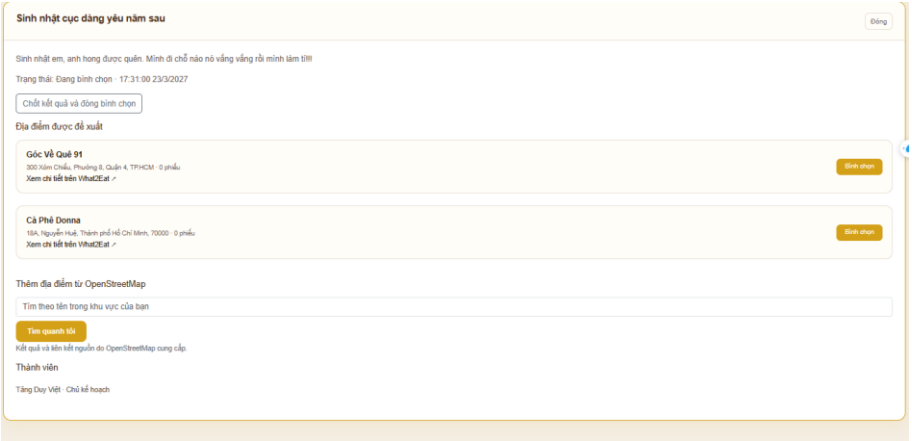
Mỗi hàng hiển thị số thành viên và số đề xuất, lấy bằng truy vấn con đếm trực tiếp. Danh sách sắp theo thời gian hẹn, kế hoạch không có lịch xếp xuống cuối.



Hình 3.20: Trang tạo kế hoạch: tên, mô tả và thời gian hẹn

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

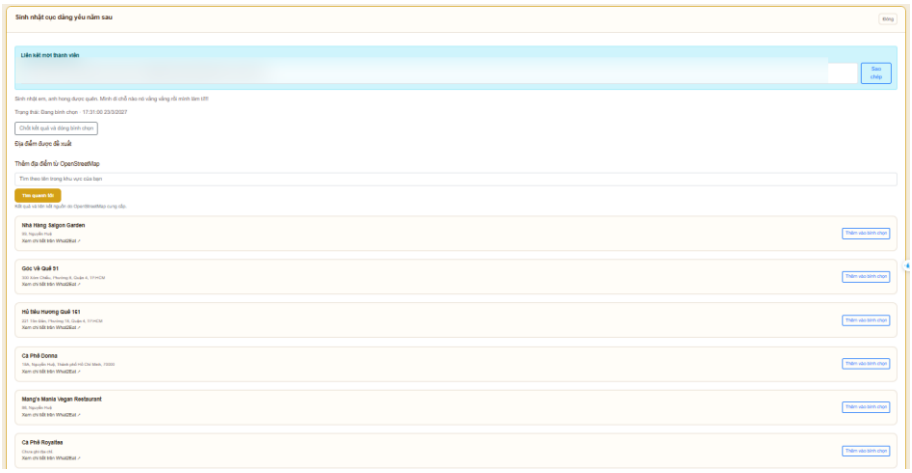
Khi tạo, máy chủ sinh một invite_token ngẫu nhiên, chỉ lưu băm SHA-256 của token trong cơ sở dữ liệu và trả liên kết mời đầy đủ một lần duy nhất trong phản hồi.



Hình 3.21: Hộp thoại xác nhận thông tin kế hoạch trước khi gửi

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Hộp thoại gom các trường đã nhập để người dùng đối chiếu lần cuối; dữ liệu chỉ được gửi đi khi người dùng bấm xác nhận, tránh tạo kế hoạch trùng do bấm đúp.



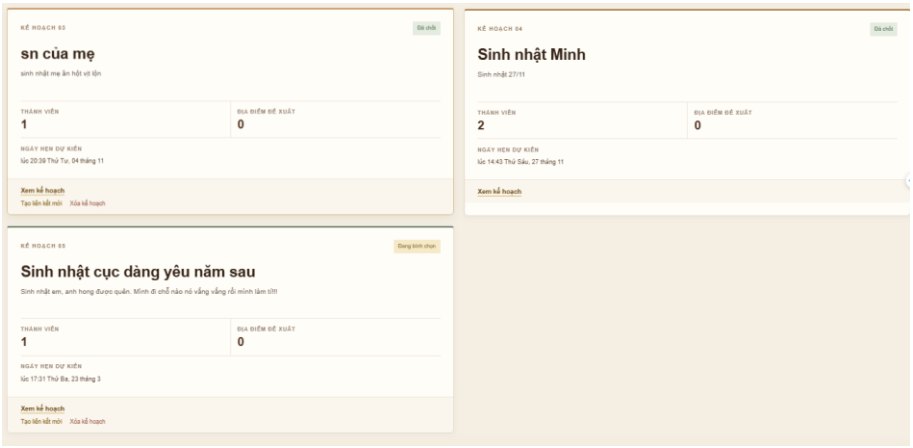
Hình 3.22: Màn hình chọn địa điểm đề xuất cho kế hoạch

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Người dùng chọn từ danh mục địa điểm đã công khai, hoặc tìm quán mới ngay trong trang bằng ô "Thêm địa điểm từ OpenStreetMap".

Mỗi đề xuất gắn với một thành viên, và một người có thể đề xuất nhiều quán.

Liên kết mời thành viên ở đầu trang chứa token truy cập nên đã được làm mờ trong ảnh.



Hình 3.23: Thẻ tổng hợp mỗi kế hoạch: trạng thái, số thành viên, số địa điểm đề xuất và ngày hẹn

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

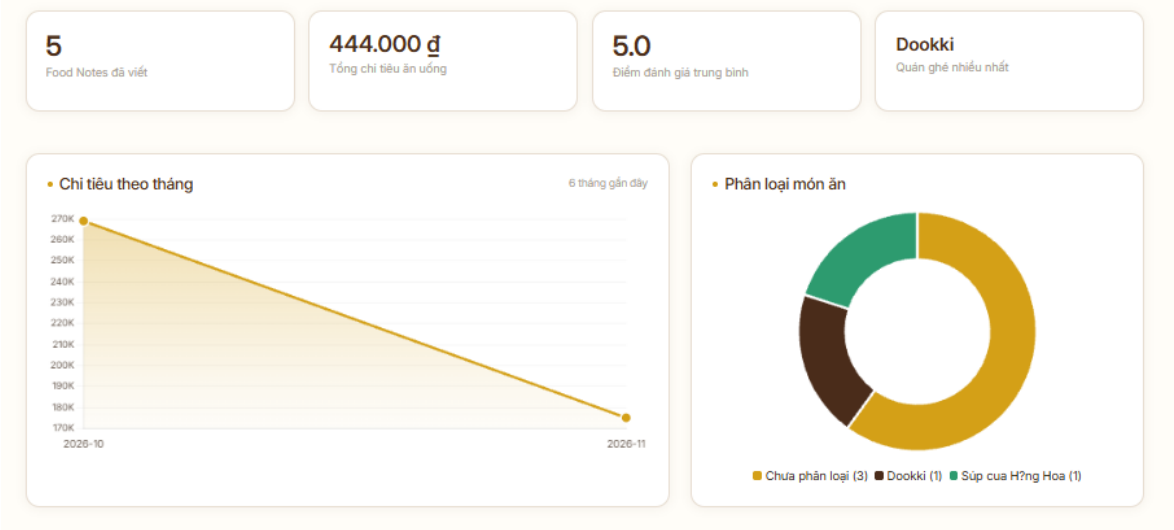
Mỗi thẻ hiển thị trạng thái (Đang bình chọn hoặc Đã chốt), số thành viên, số địa điểm đề xuất, ngày hẹn dự kiến và ba liên kết hành động.

Bên trong trang chi tiết, danh sách đề xuất được sắp theo số phiếu giảm dần, phá hoà bằng thời gian tạo.

Chủ kế hoạch có thể chuyển trạng thái sang decided hoặc closed; khi đó máy chủ từ chối mọi phiếu mới.

3.6 Thống kê cá nhân và quản trị tài khoản

Dashboard tổng hợp thói quen ăn uống của chính người dùng bằng các truy vấn GROUP BY trên bảng food_notes. Trang Hồ sơ gồm bốn thẻ: Thông tin cá nhân, Sở thích ăn uống, Bảo mật và Tài khoản.



Hình 3.24: Bảng điều khiển: tổng chi tiêu, biểu đồ theo tháng và nhóm món ăn

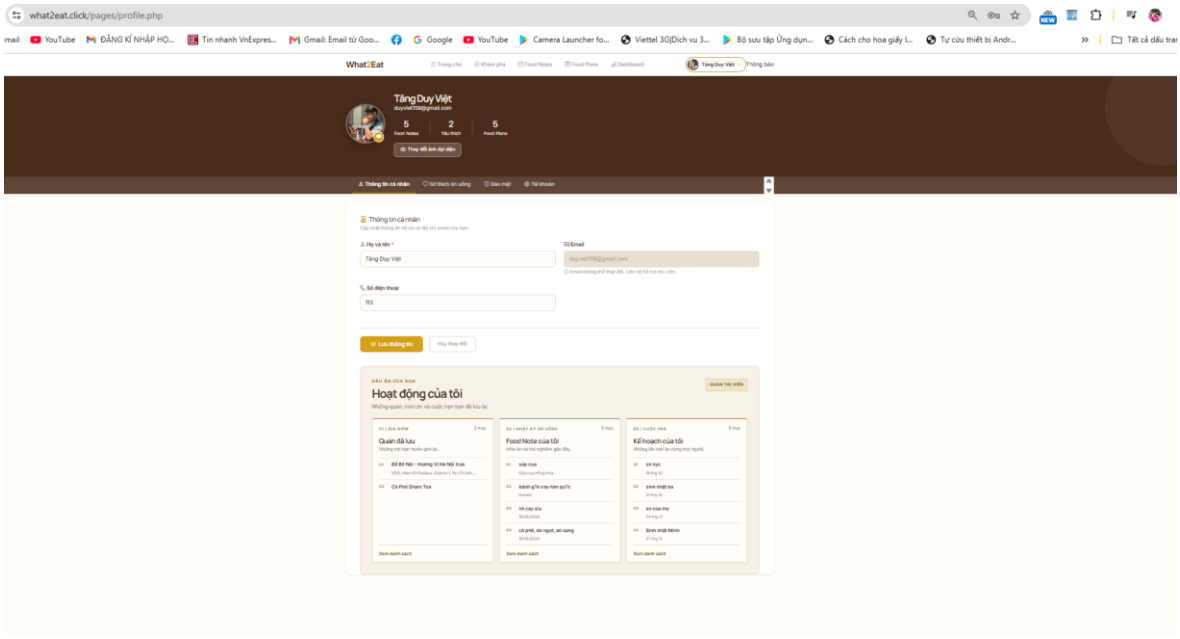
(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Bốn thẻ chỉ số lần lượt là số Food Note đã viết, tổng chi tiêu, điểm đánh giá trung bình và quán ghé nhiều nhất.

Ba giá trị đầu nằm trong một truy vấn duy nhất ở api/index.php:1164 (COUNT, SUM(price), AVG(rating) kèm số ghi chú trong tháng); tên quán dẫn đầu lấy từ danh sách top_restaurants.

Biểu đồ đường (chi tiêu theo tháng) và biểu đồ tròn (nhóm món ăn) do Chart.js vẽ từ dữ liệu JSON trả về; ảnh trên giữ lại khối chỉ số cùng hai biểu đồ này.

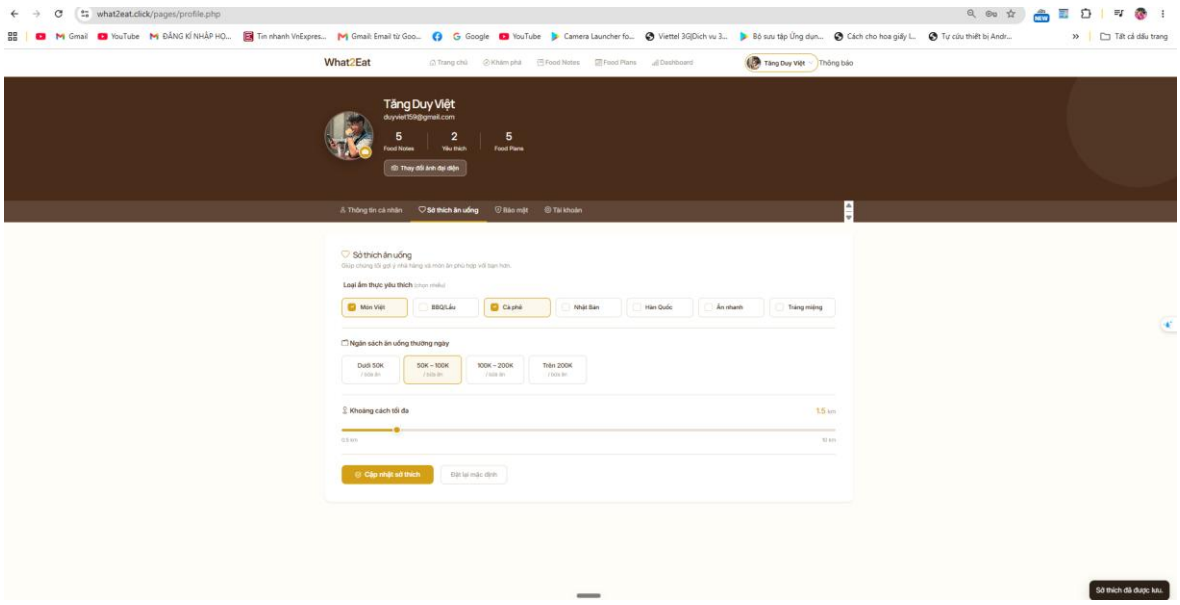
Ở bản chụp, biểu đồ cột "Quán ghé nhiều nhất" hiển thị thành một khối đen: mảng màu thanh được tính khi mảng dữ liệu còn rỗng (pages/dashboard.php:750-755), còn số liệu chỉ được nạp sau đó rồi gọi chart.update() (assets/js/site.js:791-792), nên dataset không nhận được màu hợp lệ.



Hình 3.25: Thẻ "Thông tin cá nhân" và khối "Hoạt động của tôi"

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

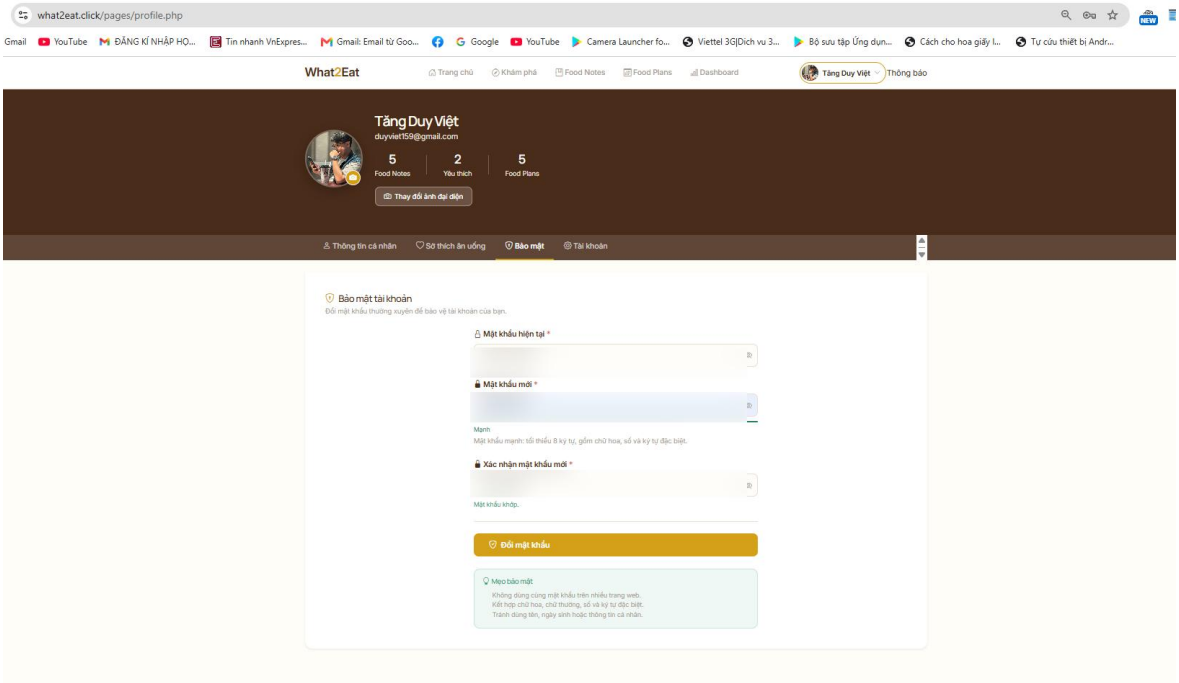
Email là trường chỉ đọc: muốn đổi phải dùng luồng hỗ trợ, vì email là khoá định danh và liên kết với token xác minh. Khối "Hoạt động của tôi" liệt kê quán đã lưu, Food Note gần đây và kế hoạch đang tham gia.



Hình 3.26: Thẻ "Sở thích ăn uống": món ưa thích, ngân sách và khoảng cách tối đa

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

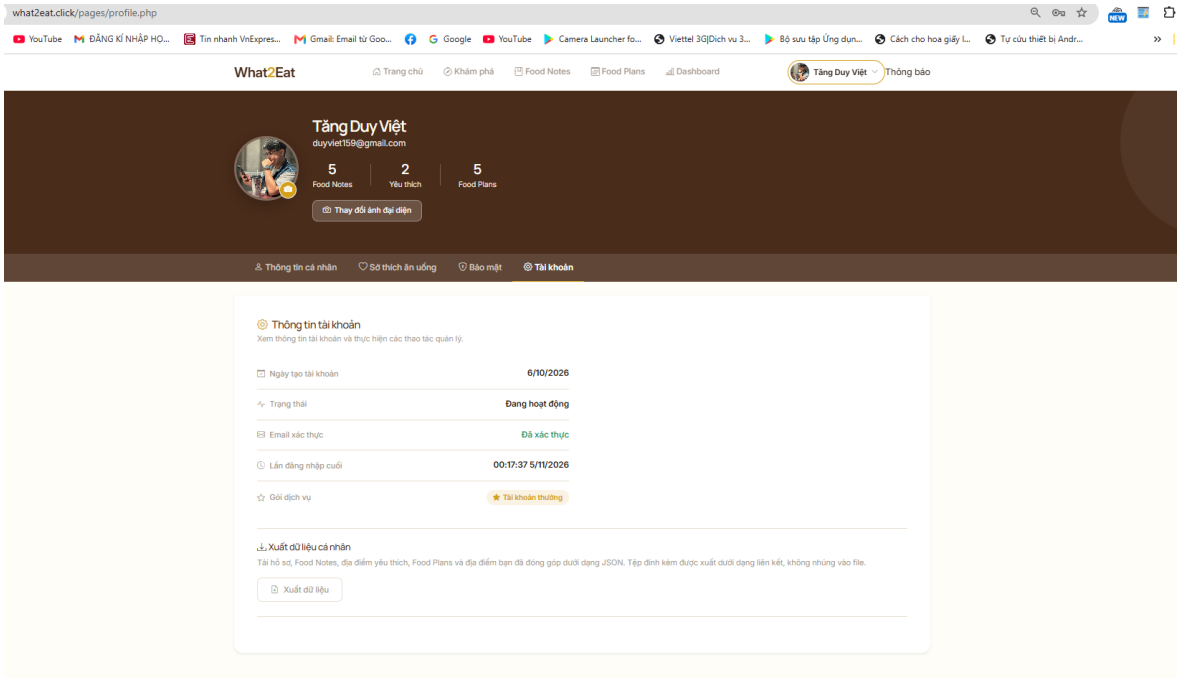
Ba nhóm sở thích lưu trong cột preferences kiểu JSON của bảng users. Khoảng cách mặc định (1.5 km) và dải giá (50K – 100K) được trang Khám phá dùng làm giá trị khởi tạo cho bộ lọc.



Hình 3.27: Thẻ "Bảo mật tài khoản" – đổi mật khẩu có kiểm tra mật khẩu cũ

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Máy chủ yêu cầu mật khẩu hiện tại, xác nhận bằng password_verify rồi mới ghi hash mới, đồng thời tăng session_version để mọi phiên đang mở của tài khoản bị vô hiệu hoá. Nội dung ô nhập đã được làm mờ trong ảnh để bảo mật.



Hình 3.28: Thẻ "Tài khoản": ngày tạo, trạng thái, gói dịch vụ và xuất dữ liệu

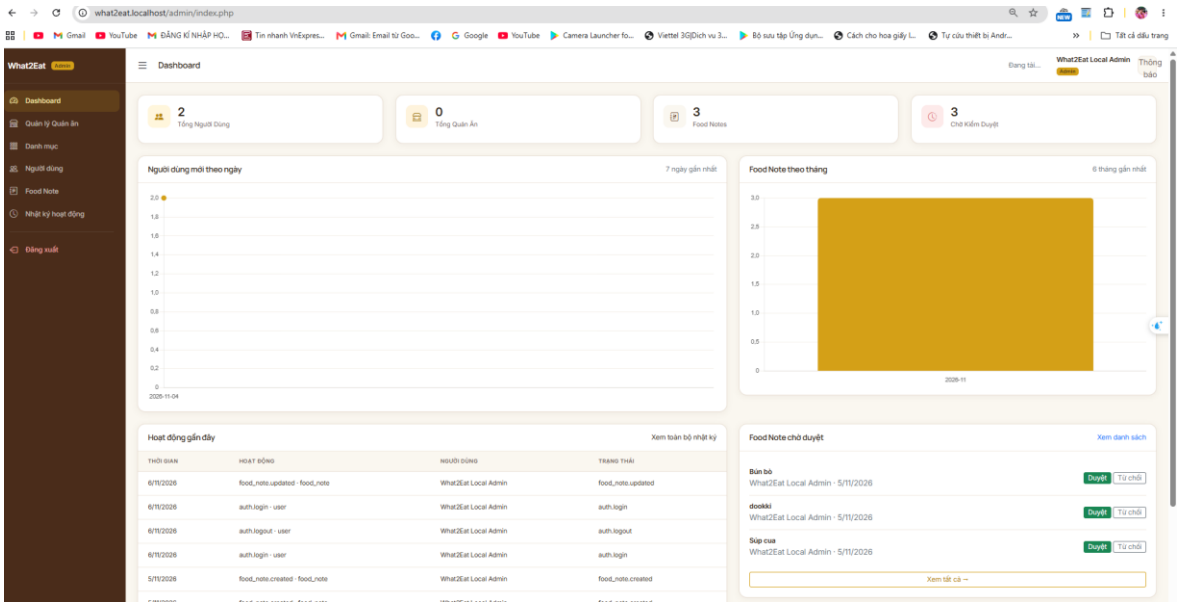
(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Nút "Xuất dữ liệu cá nhân" gộp hồ sơ, Food Note, địa điểm yêu thích, kế hoạch và đóng góp thành một tệp JSON qua liên kết tải xuống, không ghi tệp lên máy chủ.

Xoá tài khoản yêu cầu gõ đúng cụm "XÓA TÀI KHOẢN" và nhập lại mật khẩu.

3.7 Khu vực quản trị

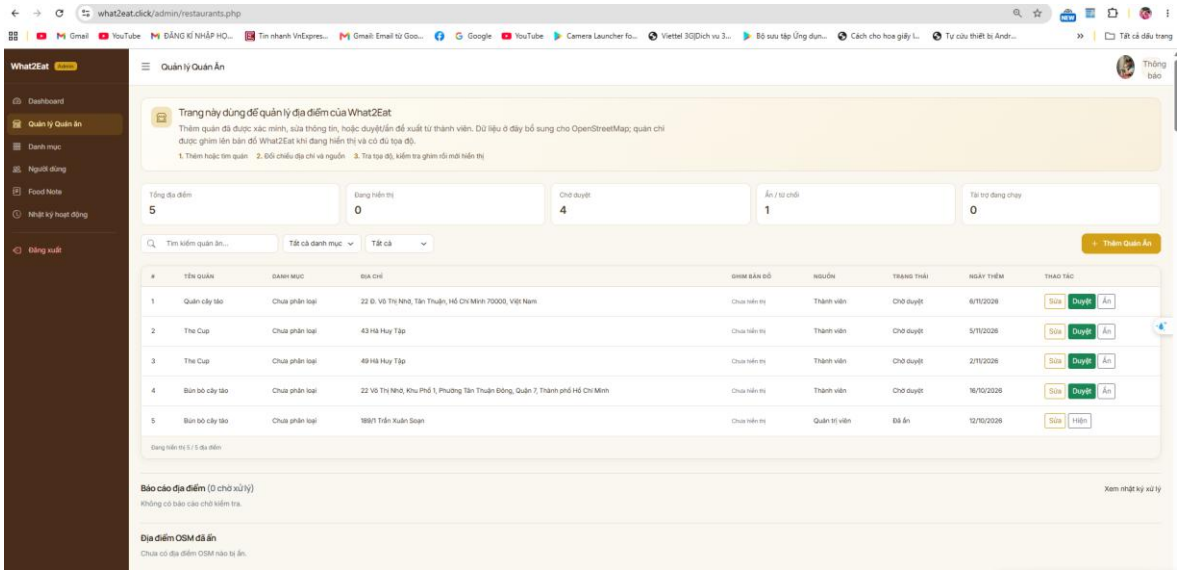
Thư mục admin/ gồm sáu trang chỉ dành cho tài khoản có role='admin'. Mọi thao tác duyệt, từ chối hay ẩn nội dung đều được ghi vào audit_logs với actor, action, entity và IP đã băm.



Hình 3.29: Bảng điều khiển quản trị: hàng chờ kiểm duyệt và các chỉ số vận hành

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

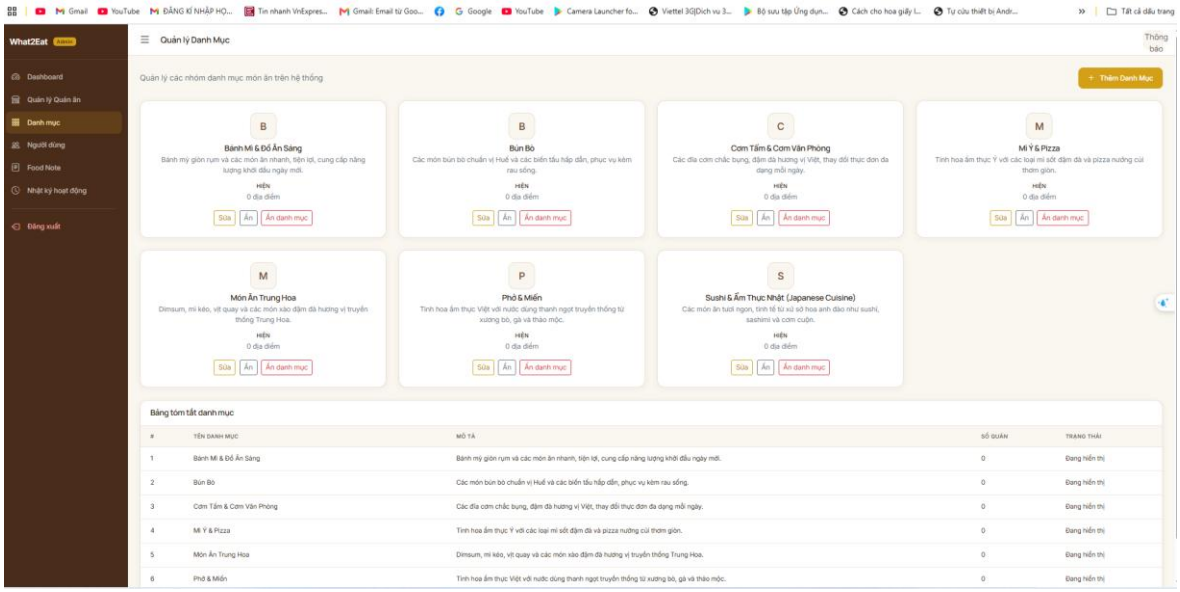
Trang tổng hợp đếm số bản ghi theo trạng thái (pending, published, rejected, hidden) để quản trị viên biết việc cần làm hôm nay, kèm liên kết tới từng hàng chờ.



Hình 3.30: Quản lý danh mục quán ăn và đề xuất của người dùng

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

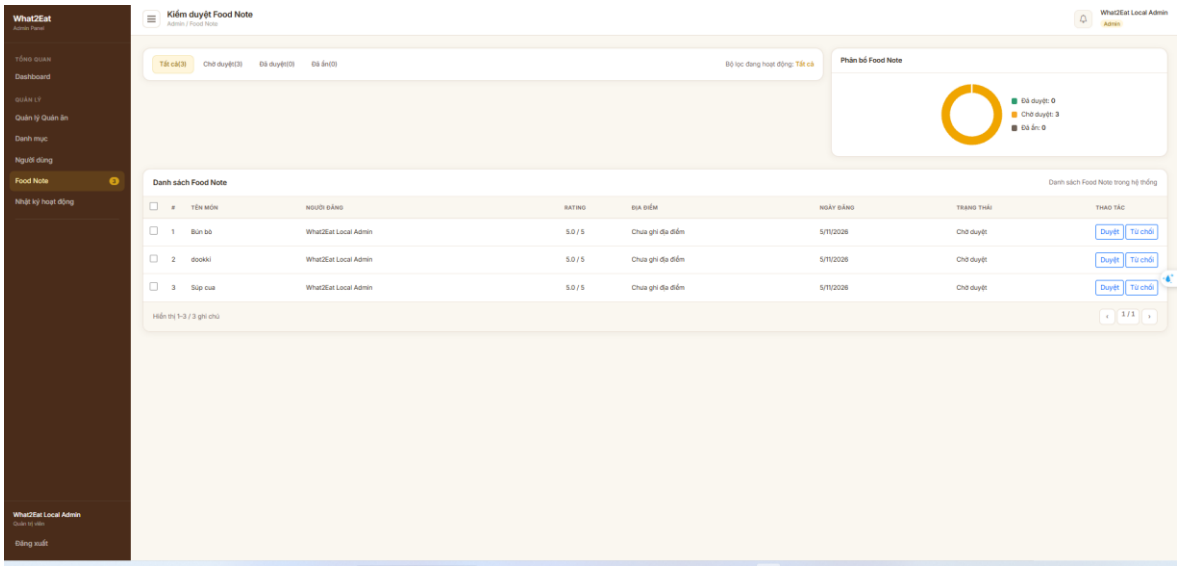
Quản trị viên lọc theo nguồn (OSM hay người dùng tạo), theo trạng thái duyệt và theo từ khoá. Duyệt một đề xuất là chuyển restaurants.status từ pending sang published; từ chối thì giữ bản ghi để đối chiếu sau này.



Hình 3.31: Quản lý danh mục món ăn

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

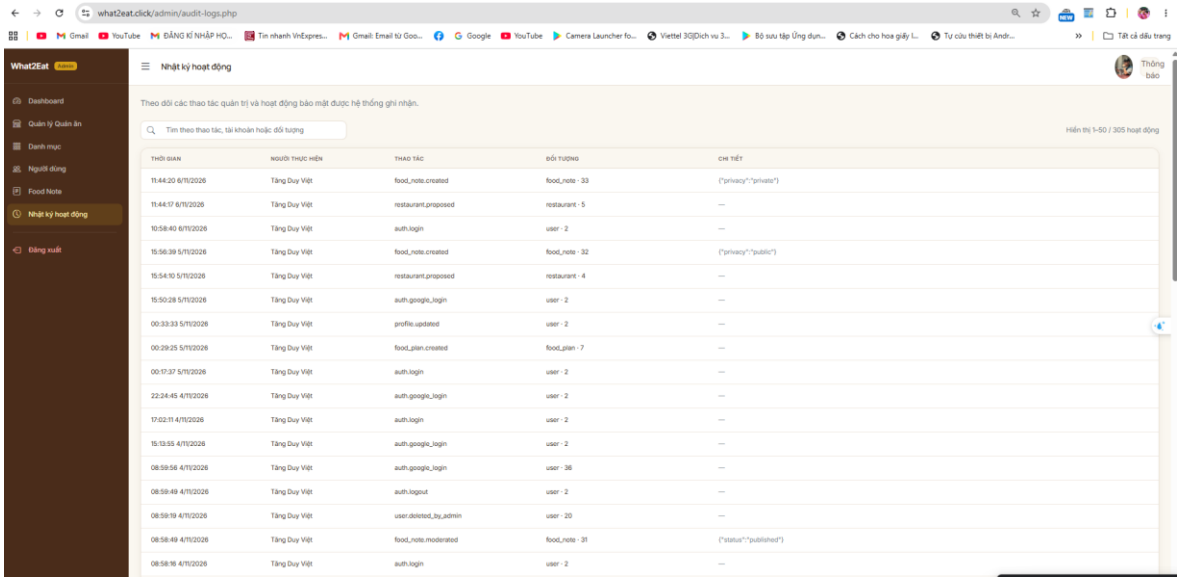
Bảng categories có slug duy nhất và khoá ngoại trỏ tới restaurants, nên một danh mục đang có quán không thể xoá tuỳ tiện; hệ thống chặn và báo lỗi rõ ràng.



Hình 3.32: Hàng chờ kiểm duyệt Food Note công khai

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Chỉ ghi chú có privacy='public' mới đi qua hàng chờ này. Nội dung nháp hoặc riêng tư của người dùng không xuất hiện ở khu vực quản trị.



Hình 3.33: Nhật ký hoạt động của hệ thống

(Nguồn: Ảnh chụp màn hình hệ thống chạy thật tại what2eat.click)

Mỗi dòng audit_logs ghi actor, hành động, loại đối tượng, mã đối tượng, metadata JSON và băm IP. Trang này cho phép lọc theo hành động và theo người thực hiện để truy vết sự vụ.

3.8 Kết luận chương 3

Bộ giao diện gồm 20 trang, chia thành ba vùng rõ rệt: vùng công khai không cần đăng nhập, vùng người dùng đã xác minh và vùng quản trị.

Cách phân vùng này khớp trực tiếp với phân quyền ở tầng API (current_user, require_admin), nên không có màn hình nào tồn tại mà không có cơ chế bảo vệ tương ứng.

Hai nguyên tắc thiết kế xuyên suốt.

Một, dữ liệu luôn có đường về nguồn (OSM, người dùng, quản trị viên) nên hiển thị được cả trạng thái kiểm duyệt.

Hai, thao tác nhạy cảm luôn có bước xác nhận: đổi mật khẩu phải nhập mật khẩu cũ, xoá tài khoản phải gõ cụm từ xác nhận.

CHƯƠNG 4. KỸ THUẬT TRIỂN KHAI VÀ LUỒNG XỬ LÝ

Chương này giải thích cách hệ thống hoạt động bên trong. Mỗi nhận định kèm vị trí trong mã nguồn theo dạng file:dòng để người đọc mở ra kiểm chứng được.

Nội dung đi theo thứ tự một yêu cầu đi qua hệ thống: khởi động, xác thực, nghiệp vụ, dữ liệu không gian, tệp tin, ngoại tuyến, vận hành và kiểm thử.

Ảnh chụp mã nguồn là nội dung nguyên văn của tệp, khoảng dòng được ghi dưới mỗi ảnh. Ở một vài đoạn mã viết dồn nhiều lệnh trên một dòng, nhóm lược ghi trong khung chữ xám và dấu … cho biết phần bị bỏ.

4.1 Khởi động và đường đi của một yêu cầu

4.1.1 Một cửa vào duy nhất

Toàn bộ API đi qua đúng một file là api/index.php. Apache rewrite mọi đường dẫn con về file này, còn PHP tự tách phần đường dẫn và phần phiên bản API ngay đầu luồng. Cũng ở đầu file, bốn header bảo mật được đặt cho mọi phản hồi.

api/index.php · dòng 8–11 · bốn header bảo mật bật cho mọi phản hồi

```
header('X-Content-Type-Options: nosniff');
header('Referrer-Policy: strict-origin-when-cross-origin');
header('X-Frame-Options: DENY');
header('Permissions-Policy: camera=(), microphone=(), geolocation=(self)');
```

Hình 4.1: Bốn header bảo mật đặt cho mọi phản hồi API

(Nguồn: api/index.php:8-11)

api/index.php · dòng 13–19 · chuẩn hoá đường dẫn về một cửa vào duy nhất

```
function route_path(): string
{
    $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
    if (str_starts_with($path, '/api/v1')) $path = substr($path, 7) ?: '/';
    elseif (str_starts_with($path, '/api')) $path = substr($path, 4) ?: '/';
    return '/' . trim($path, '/');
}
```

Hình 4.2: Hàm chuẩn hoá đường dẫn yêu cầu về một cửa vào duy nhất

(Nguồn: api/index.php:13-19)

Sau bước này, /api/v1/food-notes/12 trở thành /food-notes/12 và được so khớp bằng biểu thức chính quy trong khối dispatch bắt đầu từ dòng 679.

Cách làm này tương đương router của một micro-framework nhưng chỉ tốn vài chục dòng; đổi lại mọi route phải tự viết nên rất dễ lần theo khi debug.

4.1.2 Lớp bootstrap và kết nối cơ sở dữ liệu

api/bootstrap.php được nạp đầu tiên và đảm nhiệm năm việc: đọc biến môi trường từ .env, cấu hình phiên, mở kết nối PDO, định nghĩa các hàm tiện ích (csrf, rate_limit, audit, current_user) và đăng ký hàm xử lý lỗi. Điểm đáng học là cấu hình PDO:

api/bootstrap.php · dòng 66–75 · kết nối PDO với prepare thật, exception và mã hoá utf8mb4

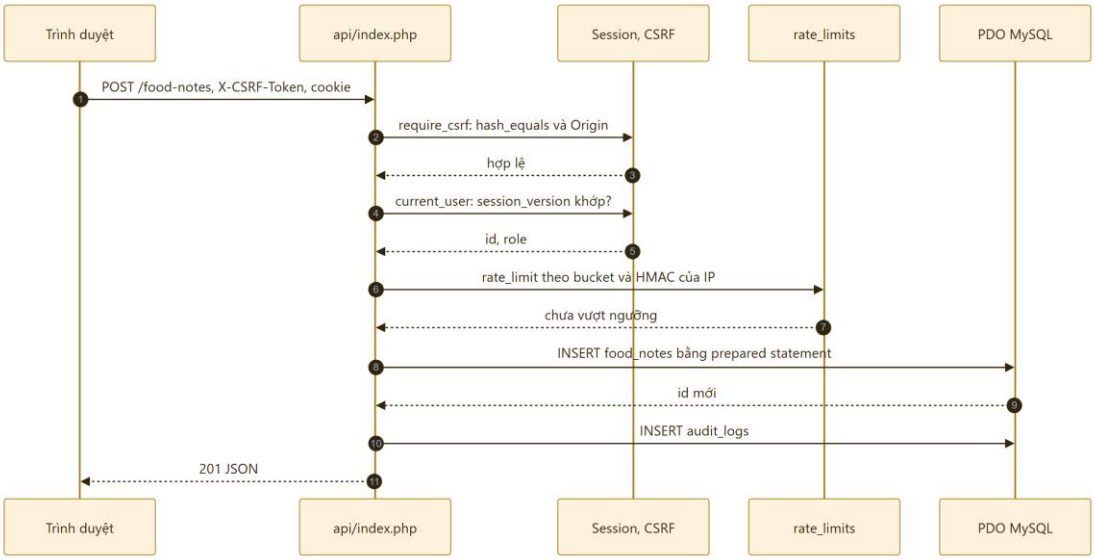
```
    $port = env_value('W2E_DB_PORT', '3306');
    $dsn = sprintf('mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4', $host, $port,
$name);
    try {
        $pdo = new PDO($dsn, $user, $password, [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
            PDO::MYSQL_ATTR_INIT_COMMAND => "SET time_zone = '+00:00'",
        ]);
        $pdo->exec('SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci');
```

Hình 4.3: Mở kết nối PDO: exception, prepared statement thật và múi giờ UTC (Nguồn: api/bootstrap.php:66-75)

ATTR_EMULATE_PREPARES = false buộc PHP dùng prepared statement thật của MySQL thay vì tự nối chuỗi rồi giả vờ escape. Đây là phòng tuyến chính chống SQL injection; mọi truy vấn trong dự án đều đi qua prepare/execute với tham số ràng buộc.

Phiên làm việc được cấu hình tập trung trong start_app_session() (bootstrap.php:83-98): use_strict_mode và use_only_cookies bật, cookie mang HttpOnly, SameSite=Lax, còn cờ Secure bật tự động khi W2E_ENV=production hoặc yêu cầu đến qua HTTPS.

4.1.3 Luồng đầy đủ của một yêu cầu có xác thực



Hình 4.4: Trình tự xử lý một yêu cầu API có ghi dữ liệu (POST /food-notes)

(Nguồn: api/index.php, api/bootstrap.php)

Bảy bước: (1) trình duyệt gửi kèm cookie phiên và header X-CSRF-Token; (2) index.php đặt security header rồi gọi bootstrap; (3) request_method và route_path xác định hành động; (4) require_csrf kiểm tra token và Origin.

(5) current_user đọc user theo session_version; (6) handler mở giao dịch, ghi bảng chính, đồng bộ nhãn, ghi audit_logs rồi commit; (7) json_response trả JSON một mức {data|message|error}.

Nguyên tắc xử lý lỗi là gom về một chỗ: ApiError mang mã HTTP và thông điệp tiếng Việt, hàm xử lý cuối đường chuyển thành JSON {"error":{"message":…}} với đúng mã trạng thái. Nhờ vậy frontend chỉ cần một hàm đọc lỗi duy nhất thay vì mỗi nơi một kiểu.

4.2 Xác thực, phân quyền và bảo vệ phiên

4.2.1 Mật khẩu và luồng đăng ký

Mật khẩu được băm bằng Argon2id – thuật toán thắng cuộc Password Hashing Competition 2015 và được OWASP khuyến nghị, thiết kế để chống tấn công bằng GPU nhờ cần cả bộ nhớ lẫn thời gian.

api/index.php · dòng 775–779 · mật khẩu chỉ được lưu dưới dạng hash Argon2id

```
        db()->beginTransaction();
        try {
            $hash = password_hash($password, defined('PASSWORD_ARGON2ID') ?
PASSWORD_ARGON2ID : PASSWORD_DEFAULT);
            $insert = db()->prepare("INSERT INTO
users(name,email,password_hash,preferences,role,status)
VALUES(?,?,?,?,'user','pending')");
            $insert->execute([$name, $email, $hash,
json_encode(array_slice(array_map('strval', (array)($body['preferences'] ?? [])), 0,
```

Hình 4.5: Băm mật khẩu bằng Argon2id rồi chèn hàng users trong cùng một giao dịch

(Nguồn: api/index.php:775-779)

Ba chi tiết đáng chú ý của luồng đăng ký.

Tài khoản sinh ra ở trạng thái pending nên chưa đăng nhập được; index.php:812 chặn bằng mã 403 "Hãy xác nhận email trước khi đăng nhập".

Vi phạm khoá UNIQUE email được bắt bằng SQLSTATE 23000 và dịch sang mã 409, thay vì để lỗi PHP lộ ra ngoài.

Toàn bộ nằm trong một giao dịch cùng bảng email_tokens, nên không sinh token cho một tài khoản ghi dở.

Token xác minh không lưu nguyên văn. Hệ thống chỉ lưu băm SHA-256 (hàm token_hash) và so khớp lúc sử dụng, nên dù bảng email_tokens bị đọc lén cũng không suy ra được liên kết hợp lệ.

Khi dùng, truy vấn có FOR UPDATE để hai lần bấm liên kết không cùng kích hoạt tài khoản.

4.2.2 Phiên làm việc và thu hồi hàng loạt

Đăng nhập thành công gọi session_regenerate_id(true) để chống session fixation, rồi lưu cả user_id lẫn session_version vào phiên (index.php:817). Mỗi lần gọi current_user, máy chủ đọc lại user từ CSDL và so session_version trong phiên với cột tương ứng (bootstrap.php:208); lệch là xoá phiên và trả 401.

Cơ chế này giải quyết bài toán mà cookie thông thường không làm được: buộc các thiết bị đang đăng nhập phải đăng nhập lại sau khi đổi mật khẩu, admin khoá tài khoản, hoặc người dùng tự xoá tài khoản.

Muốn thu hồi, chỉ cần tăng session_version trong bảng users mà không phải quét bảng phiên.

4.2.3 Chống CSRF

Hàm require_csrf() chạy trước mọi thao tác ghi và đặt hai lớp kiểm tra liền nhau.

api/bootstrap.php · dòng 184–195 · hai lớp kiểm tra CSRF: token và Origin

```
function require_csrf(): void
{
    $token = (string)($_SERVER['HTTP_X_CSRF_TOKEN'] ?? '');
    if (empty($_SESSION['csrf']) || $token === '' ||
!hash_equals((string)$_SESSION['csrf'], $token)) {
        throw new ApiError(419, 'Phiên đã hết hạn. Hãy tải lại trang rồi thử lại.');
    }
    $origin = $_SERVER['HTTP_ORIGIN'] ?? '';
    if ($origin !== '') {
        $configuredBase = env_value('W2E_BASE_URL');
        $expected = $configuredBase ? rtrim($configuredBase, '/') :
((((!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ? 'https://' : 'http://')
. ($_SERVER['HTTP_HOST'] ?? ''));
        if (!hash_equals(strtolower($expected), strtolower(rtrim($origin, '/')))) throw
new ApiError(403, 'Nguồn yêu cầu không hợp lệ.');
    }
```

Hình 4.6: Hai lớp kiểm tra CSRF: token lưu trong phiên và Origin của yêu cầu (Nguồn: api/bootstrap.php:184-195)

Lớp thứ nhất là token ngẫu nhiên 32 byte lưu trong phiên và gửi qua header tùy chọn; cách này an toàn hơn form field vì một form HTML bên ngoài không thể tự động đính kèm.

Lớp thứ hai là so Origin với W2E_BASE_URL.

Cả hai đều dùng hash_equals thay vì ==, để thời gian so sánh không phụ thuộc nội dung, chặn tấn công dò từng byte.

4.2.4 Giới hạn tần suất

Bảng rate_limits lưu số lần theo bộ (bucket) và chủ thể. Chủ thể không phải IP thô mà là HMAC-SHA256 của IP với W2E_APP_SECRET (bootstrap.php:228-232), nên nhật ký không chứa dữ liệu định danh cá nhân trực tiếp.

api/bootstrap.php · dòng 234–244 · giới hạn tần suất cửa sổ cố định theo IP đã băm

```
function rate_limit(string $bucket, int $maximum, int $windowSeconds): void
{
    $now = time();
    $start = gmdate('Y-m-d H:i:s', intdiv($now, $windowSeconds) * $windowSeconds);
    $subject = client_ip_hash();
    $statement = db()->prepare('INSERT INTO
rate_limits(bucket,subject_hash,window_start,hits) VALUES(?,?,?,1) ON DUPLICATE KEY
UPDATE hits=hits+1');
    $statement->execute([$bucket, $subject, $start]);
    $check = db()->prepare('SELECT hits FROM rate_limits WHERE bucket=? AND
subject_hash=? AND window_start=?');
    $check->execute([$bucket, $subject, $start]);
    if ((int)$check->fetchColumn() > $maximum) throw new ApiError(429, 'Bạn thao tác
quá nhanh. Hãy thử lại sau ít phút.');
}
```

Hình 4.7: Hàm rate_limit(): cửa sổ thời gian cố định và ngưỡng vượt

(Nguồn: api/bootstrap.php:234-244)

Cửa sổ là cố định (fixed window) thay vì trượt, đổi lại chỉ cần một dòng INSERT … ON DUPLICATE KEY UPDATE nên không có tranh chấp ghi.

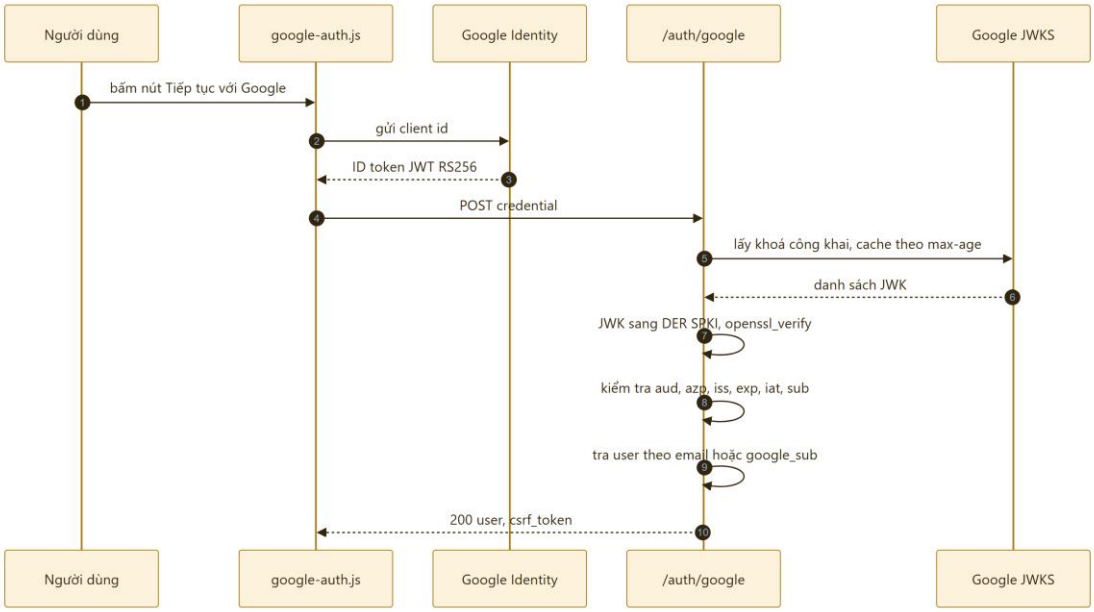
Hệ quả là ở ranh giới hai cửa sổ có thể vượt gấp đôi ngưỡng. Mức này chấp nhận được với một ứng dụng phục vụ cộng đồng và đã được ghi ở mục hạn chế.

Bảng 4.1: Các bộ giới hạn tần suất được khai báo trong mã nguồn

Bucket	Ngưỡng	Cửa sổ	Vị trí khai báo
register	5 (100 ở local)	3600 s	api/index.php:769
login	10	900 s	api/index.php:809
google_login	20	900 s	api/index.php:824
forgot / reset	4 / 8	3600 s	api/index.php:889, 899
resend_verify (+ theo phút)	3 và 1	3600 s / 60 s	api/index.php:803

osm (quét Overpass)	20	60 s	api/index.php:487
place_report	10	3600 s	api/index.php:972
corrections	10	3600 s	api/completion.php:120
account_delete	3	3600 s	api/index.php:1153
admin_user_delete	20 (100 ở local)	3600 s	api/index.php:1284

4.2.5 Đăng nhập Google và việc tự viết bộ kiểm tra JWT



Hình 4.8: Trình tự xác minh ID token của Google Identity Services

(Nguồn: api/index.php:824-880, api/google-auth.php)

Dự án không dùng thư viện OAuth. api/google-auth.php (158 dòng) tự làm bốn việc: giải mã base64url phân đoạn header/claims/signature; dựng khoá công khai từ JWK sang định dạng DER/SPKI để openssl_verify hiểu được; lấy và đệm bộ khoá JWKS theo kid; kiểm tra các claim bắt buộc.

api/google-auth.php · dòng 135–156 · kiểm kid, chữ ký RS256 và toàn bộ claim bắt buộc

```
    if (!isset($keys[$header['kid']])) $keys = google_signing_keys($clientId, true);
    $jwk = $keys[$header['kid']] ?? null;
    $publicKey = is_array($jwk) ? google_jwk_public_key($jwk) : null;
    if (!$publicKey) throw new ApiError(401, 'ID token Google không hợp lệ.');
    $verified = openssl_verify($encodedHeader . '.' . $encodedClaims, $signature,
$publicKey, OPENSSL_ALGO_SHA256);
    if ($verified !== 1) throw new ApiError(401, 'ID token Google không hợp lệ.');

    $now = time();
    $audience = $claims['aud'] ?? null;
    $audienceMatches = is_string($audience) && hash_equals($clientId, $audience);
    if (is_array($audience)) {
        $audienceMatches = in_array($clientId, $audience, true) &&
isset($claims['azp']) && is_string($claims['azp']) && hash_equals($clientId,
$claims['azp']);
    }
    if (!$audienceMatches || (isset($claims['azp']) && (!is_string($claims['azp']) ||
!hash_equals($clientId, $claims['azp'])))
        || !in_array($claims['iss'] ?? null,
['accounts.google.com','https://accounts.google.com'], true)
        || !is_numeric($claims['exp'] ?? null) || (int)$claims['exp'] < $now
        || !is_numeric($claims['iat'] ?? null) || (int)$claims['iat'] > $now + 60
        || !is_string($claims['sub'] ?? null) || $claims['sub'] === '' ||
strlen($claims['sub']) > 255
        || !is_string($claims['email'] ?? null) || !filter_var($claims['email'],
FILTER_VALIDATE_EMAIL)
        || !in_array($claims['email_verified'] ?? null, [true, 'true'], true)) {
        throw new ApiError(401, 'ID token Google không hợp lệ hoặc đã hết hạn.');
    }
```

Hình 4.9: Kiểm tra kid, chữ ký RS256 và toàn bộ claim bắt buộc của ID token (Nguồn: api/google-auth.php:135-156)

Ba quyết định bảo mật đáng chú ý.

Một, nếu kid lạ xuất hiện, bộ đệm khoá bị làm mới ngay thay vì tin luôn.

Hai, aud được kiểm tra cả dạng chuỗi lẫn dạng mảng, kèm điều kiện azp (google-auth.php:144-148).

Ba, iss, exp, iat, sub và email_verified đều được đối chiếu; token thiếu email_verified bị từ chối.

Ở tầng ứng dụng, index.php:846 còn chặn trường hợp mạo danh qua tên miền: email chỉ được coi là đáng tin khi thuộc gmail.com hoặc khi hosted_domain khớp email.

Dòng 850 từ chối việc liên kết một tài khoản đã gắn google_sub với một subject khác, tránh chiếm tài khoản qua đổi email.

4.2.6 Ma trận phân quyền

Bảng 4.2: So sánh ba nhóm chủ thể theo các khả năng nghiệp vụ

Khả năng	Khách	Người dùng	Quản trị
Xem trang chủ, địa điểm công khai	Có	Có	Có
Quét Overpass theo khu vực	Có (có giới hạn)	Có	Có
Tạo / sửa / xoá Food Note của mình	Không	Có	Có
Công khai ghi chú (phải duyệt)	Không	Có	Có
Yêu thích, kế hoạch nhóm, bình chọn	Không	Có	Có
Duyệt nội dung, ẩn / xoá bản ghi	Không	Không	Có
Truy cập thư mục admin/	Không	Không	Có
Xem audit_logs	Không	Không	Có
Tự xoá tài khoản	Không	Có	Không (bị chặn)

Phân quyền chỉ có hai hàm cửa: current_user() (bootstrap.php:198-219) và require_admin() (bootstrap.php:221-226). Không có ngoại lệ phân quyền theo trang, nên thêm route mới bắt buộc chọn một trong hai – đây là lý do dự án không phát sinh lỗ hổng "quên kiểm tra quyền" ở từng file.

4.3 Khai thác dữ liệu không gian OpenStreetMap

Đây là phần khó nhất của đề tài vì phụ thuộc dịch vụ của bên thứ ba vốn chậm, hay quá tải và có chính sách sử dụng nghiêm ngặt.

Hệ thống dùng ba dịch vụ: Overpass API truy vấn đối tượng theo toạ độ, Nominatim chuyển giữa địa chỉ và toạ độ, máy chủ tile vẽ nền bản đồ.

4.3.1 Câu truy vấn Overpass

Câu truy vấn được dựng động theo bán kính và danh mục tag. Điều kiện loại trừ rất dài nhưng cần thiết, vì OSM lưu cả những quán đã đóng cửa dưới dạng tag lifecycle.

api/index.php · dòng 513–522 · sinh truy vấn Overpass QL và giới hạn kết quả

```
    foreach ([
        'amenity' =>
'restaurant|cafe|fast_food|food_court|ice_cream|pub|bar|biergarten|canteen',
        'shop' => 'bakery|pastry|confectionery|tea',
    ] as $tagKey => $tagValues) {
        $lifecycle = '["disused"!="yes"]["abandoned"!="yes"]["demolished"!="yes"]
["disused:' . $tagKey . '"!~"."]["abandoned:' . $tagKey . '"!~"."]["demolished:' .
$tagKey . '"!~"."]';
        foreach (['node', 'way', 'relation'] as $entity) {
            $blocks[] = $entity . '["' . $tagKey . '"~"^(' . $tagValues . ')$"' .
$lifecycle . '](around:' . $radius . ',' . $latKey . ',' . $lonKey . ');';
        }
    }
    $query = '[out:json][timeout:12];(' . implode('', $blocks) . ');out center 250;';
```

Hình 4.10: Sinh truy vấn Overpass QL theo tag, bán kính và giới hạn kết quả (Nguồn: api/index.php:513-522)

Ba tham số có chủ đích: timeout:12 để server không treo, out center vì đối tượng way (quán ăn thường là polygon) không có toạ độ gốc, và giới hạn 250 kết quả để phản hồi nằm trong kích thước xử lý được ở trình duyệt.

4.3.2 Đua ba máy chủ Overpass

api/index.php · dòng 523–533 · danh sách máy chủ và khởi tạo curl_multi

```
    $endpoints = [
        'https://maps.mail.ru/osm/tools/overpass/api/interpreter',
        'https://overpass.private.coffee/api/interpreter',
        'https://overpass-api.de/api/interpreter',
    ];
    $success = null;
    if (function_exists('curl_multi_init')) {
        $multi = curl_multi_init(); $handles = [];
        foreach ($endpoints as $endpoint) {
            $ch = curl_init($endpoint);
            curl_setopt_array($ch, [CURLOPT_POST => true, CURLOPT_POSTFIELDS =>
http_build_query(['data' => $query]), CURLOPT_RETURNTRANSFER => true,
CURLOPT_CONNECTTIMEOUT => 4, CURLOPT_TIMEOUT => 9, CURLOPT_HTTPHEADER => ['Accept:
application/json','User-Agent: What2Eat/1.0 (OpenStreetMap community data)'],
CURLOPT_FOLLOWLOCATION => false]);
```

Hình 4.11: Danh sách ba máy chủ Overpass và khởi tạo curl_multi

(Nguồn: api/index.php:523-533)

api/index.php · dòng 536–551 · vòng polling lấy phản hồi đầu tiên hợp lệ rồi break 2

```
        $running = 0;
        do {
            $status = curl_multi_exec($multi, $running);
            while (($info = curl_multi_info_read($multi)) !== false) {
                $ch = $info['handle']; $raw = curl_multi_getcontent($ch); $code =
(int)curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
                if ($code === 200 && is_string($raw)) {
                    $data = json_decode($raw, true);
                    if (is_array($data) && isset($data['elements']) &&
is_array($data['elements'])) $success = $data;
                }
                curl_multi_remove_handle($multi, $ch); curl_close($ch);
                $handles = array_values(array_filter($handles, static fn($handle) =>
$handle !== $ch));
                if ($success !== null) break 2;
            }
            if ($running > 0 && curl_multi_select($multi, 0.1) === -1) usleep(100000);
        } while ($running > 0 && $status === CURLM_OK);
        foreach ($handles as $ch) {
```

Hình 4.12: Vòng polling nhận phản hồi đầu tiên hợp lệ rồi thoát bằng break 2

(Nguồn: api/index.php:536-551)

curl_multi cho phép gửi cả ba yêu cầu cùng lúc và nhận kết quả theo thứ tự hoàn thành.

break 2 thoát khỏi cả vòng lặp polling lẫn vòng xử lý thông tin khi có phản hồi đầu tiên đạt chuẩn; phần còn lại được đóng ở dòng 551-554.

Thời gian chờ thực tế bằng máy nhanh nhất, không phải tổng ba máy.

Nếu PHP không có curl_multi, nhánh elseif ở dòng 555 thử tuần tự từng máy với timeout ngắn hơn.

4.3.3 Bốn tầng dự phòng

Toàn bộ cơ chế dự phòng được tóm tắt ở sơ đồ tại mục 2.5. Bảng dưới đây quy chiếu các tầng bộ nhớ đệm và chính sách tuổi dữ liệu tương ứng.

Bảng 4.3: Các tầng bộ nhớ đệm và chính sách tuổi dữ liệu

Tầng	**Thời hạn**	**Nơi lưu**	**Hành vi khi hết hạn**
Kết quả Overpass	600 s	Bảng api_cache (MySQL)	Gọi lại Overpass, giữ bản cũ tới 86.400 s
Ảnh tile bản đồ	604.800 s	storage/cache/osm-tiles/z/x/y.png	Thử hai máy chủ tile, nếu vẫn lỗi trả ảnh cũ
Địa chỉ Nominatim	30 ngày	Bảng api_cache	Bỏ qua, trả kết quả không có địa chỉ
Danh mục địa điểm	Vĩnh viễn (import)	Bảng place_catalog	Cần chạy lại scripts/build-public-snapshot.mjs
Shell PWA	Theo version tên cache	Cache Storage của trình duyệt	activate xoá cache mang tên cũ (sw.js:24-28)

Chính sách "stale" là điểm tinh tế nhất: api/index.php:574 cho phép dùng bản cũ tới 24 giờ khi dịch vụ ngoài lỗi, còn serve_osm_tile() ở dòng 616-618 trả ảnh cũ nếu cả hai máy chủ tile đều không phản hồi.

Người dùng thấy dữ liệu hơi cũ thay vì thấy trang trắng – đánh đổi hợp lý với một ứng dụng bản đồ.

4.3.4 Proxy tile và tôn trọng chính sách sử dụng

api/index.php · dòng 592–597 · phục vụ tile từ đĩa và danh sách máy chủ nguồn

```
    if (is_string($cachedTile) && $cachedTile !== '' && $cacheAge <= 604800) {
        header('Content-Type: image/png'); header('Cache-Control: public, max-
age=604800, stale-while-revalidate=86400'); header('X-Content-Type-Options: nosniff');
echo $cachedTile; exit;
    }

    $urls = ["https://a.tile.openstreetmap.fr/hot/$z/$x/$y.png",
"https://tile.openstreetmap.org/$z/$x/$y.png"];
    foreach ($urls as $url) {
```

Hình 4.13: Phục vụ tile đã đệm từ đĩa và danh sách máy chủ tile nguồn

(Nguồn: api/index.php:592-597)

Trước khi đệm, mỗi tile đi thẳng tới máy chủ OSM công cộng – vi phạm tinh thần tile usage policy, vì một phiên người dùng tạo hàng chục request.

Sau khi đệm, một tile chỉ được tải lên máy chủ nguồn nhiều nhất một lần mỗi 7 ngày, tính cho toàn bộ người dùng.

Ghi tệp dùng mô hình ghi-tạm-rồi-đổi-tên (dòng 605-611) để không bao giờ đọc phải một tệp đang ghi dở.

4.3.5 Nominatim và khoá phân tán trong MySQL

Nominatim áp giới hạn một yêu cầu mỗi giây cho mỗi client. Thách thức ở đây là máy chủ PHP chạy nhiều tiến trình đồng thời, nên "một giây" phải tính trên toàn ứng dụng chứ không trên từng tiến trình. Lời giải là dùng chính MySQL làm cái khoá.

api/completion.php · dòng 60–63 · GET_LOCK toàn cục và nhịp đệm 1,1 giây

```
    $lock=db()->query("SELECT GET_LOCK('w2e_nominatim_rate_limit',15)");if((int)$lock-
>fetchColumn()!==1)return $p;
    try{
        $clockKey=hash('sha256','nominatim:global-last-request');$q=db()-
>prepare('SELECT body FROM api_cache WHERE cache_key=?');$q-
>execute([$clockKey]);$last=(float)json_decode((string)$q-
>fetchColumn(),true);$wait=1100000-(int)
((microtime(true)-$last)*1000000);if($wait>0)usleep($wait);
        $save=db()->prepare('INSERT INTO api_cache(cache_key,body,cached_at,expires_at)
VALUES(?,?,UTC_TIMESTAMP(),DATE_ADD(UTC_TIMESTAMP(),INTERVAL 30 DAY)) ON DUPLICATE KEY
UPDATE
body=VALUES(body),cached_at=VALUES(cached_at),expires_at=VALUES(expires_at)');$save-
```

Hình 4.14: GET_LOCK toàn cục và nhịp đệm 1,1 giây trước khi gọi Nominatim (Nguồn: api/completion.php:60-63)

GET_LOCK cho phép tối đa 15 giây chờ; nếu không lấy được khoá, hàm trả về ngay kèm dữ liệu chưa có địa chỉ, thay vì xếp hàng vô hạn.

Con số 1.100.000 microsecond tạo biên độ an toàn 10% so với ngưỡng 1 giây.

Địa chỉ suy ra từ reverse geocoding được đánh dấu rõ là "Địa chỉ gần tọa độ · Nominatim" (dòng 59 và 70) để không ai nhầm đó là số nhà đã xác minh.

4.3.6 Lọc theo khoảng cách: bounding box trước, Haversine sau

Tính Haversine cho toàn bộ bảng sẽ lần lượt quét từng dòng và không dùng được index.

Dự án chia làm hai bước: dựng một hộp bao quanh tâm tìm để lọc bằng BETWEEN trên cột toạ độ (dùng được index ix_restaurants_coords), rồi mới tính khoảng cách cầu thật sự trên tập nhỏ.

api/index.php · dòng 934–941 · công thức Haversine viết ngay trong SQL

```
            $sql = "SELECT r.*,c.name AS category_name,
                    EXISTS(SELECT 1 FROM restaurant_sponsorships s WHERE
s.restaurant_id=r.id AND s.status='active' AND s.starts_at<=UTC_TIMESTAMP() AND
s.ends_at>UTC_TIMESTAMP()) AS sponsored,
                    (6371000 * 2 * ASIN(SQRT(LEAST(1,
                      POW(SIN(RADIANS(r.latitude - ?) / 2), 2) +
                      COS(RADIANS(?)) * COS(RADIANS(r.latitude)) *
POW(SIN(RADIANS(r.longitude - ?) / 2), 2)
                    )))) AS distance_meters
                    FROM restaurants r LEFT JOIN categories c ON c.id=r.category_id
                    WHERE {$where} HAVING distance_meters <= ? ORDER BY
distance_meters,r.name LIMIT 250";
```

Hình 4.15: Truy vấn tìm quán gần nhất với Haversine viết trực tiếp trong SQL (Nguồn: api/index.php:934-941)

LEAST(1, …) chặn lỗi làm tròn khiến ASIN nhận giá trị lớn hơn 1 và trả NULL.

Dùng HAVING thay vì WHERE là bắt buộc, vì MySQL không cho lọc trên cột vừa tính trong WHERE.

Bán kính trái phải còn được hiệu chỉnh khi hộp bao vượt kinh tuyến 180 (dòng 929-931). Chi tiết nhỏ, nhưng thiếu nó thì tìm kiếm ở Thái Bình Dương sẽ trả rỗng.

Cùng công thức xuất hiện ở ba nơi với vai trò khác nhau: SQL cho lọc phía máy chủ (index.php:936), PHP cho làm giàu kết quả OSM (index.php:327-346 và 377-393), JavaScript cho sắp xếp tức thời ở trình duyệt (assets/js/site.js:330-332, restaurant-map.js:109).

Bán kính Trái Đất dùng chung là 6.371 km. PHP dùng asin, JavaScript dùng atan2 – hai dạng tương đương của cùng một công thức.

4.3.7 Tìm kiếm không phân biệt dấu

assets/js/site.js · dòng 46 · slug hoá tiếng Việt không phân biệt dấu

```
  const normalizeText = (value) => String(value||'').normalize('NFD').replace(/[\u0300-
\u036f]/g,'').replace(/đ/g,'d').toLocaleLowerCase('vi');
```

Hình 4.16: Hàm normalizeText(): slug hoá tiếng Việt để tìm kiếm không phân biệt dấu

(Nguồn: assets/js/site.js:46)

Chuẩn Unicode NFD tách nguyên âm và dấu thanh thành hai mã ký tự riêng, nên chỉ cần xoá dải U+0300–U+036F là mất hết dấu.

Riêng đ/Đ không có dạng tổ hợp nên phải thay thủ công. Người dùng gõ "pho", "cafe" hay "Quan an vat" đều tìm ra kết quả.

Máy chủ cũng có đường tương tự dùng iconv ASCII//TRANSLIT (index.php:450-451) cho dữ liệu OSM có ký tự ngoài tiếng Việt.

4.3.8 Bộ dữ liệu nền 8.600 địa điểm

Vì Overpass công cộng không đủ nhanh cho mọi lượt tải trang, dự án import một snapshot TP. Hồ Chí Minh vào bảng place_catalog và phục vụ trực tiếp từ CSDL.

Tệp assets/data/hcm-food-places.json chứa 8.600 phần tử, nguồn ghi rõ "OpenStreetMap via Geofabrik Vietnam snapshot and district data supplied to What2Eat".

Snapshot lấy ngày 15/10/2026, import ngày 03/11/2026 với 3.450 bản ghi thêm mới, 86 bản trùng bỏ qua và 1.000 bản sửa lại kinh độ.

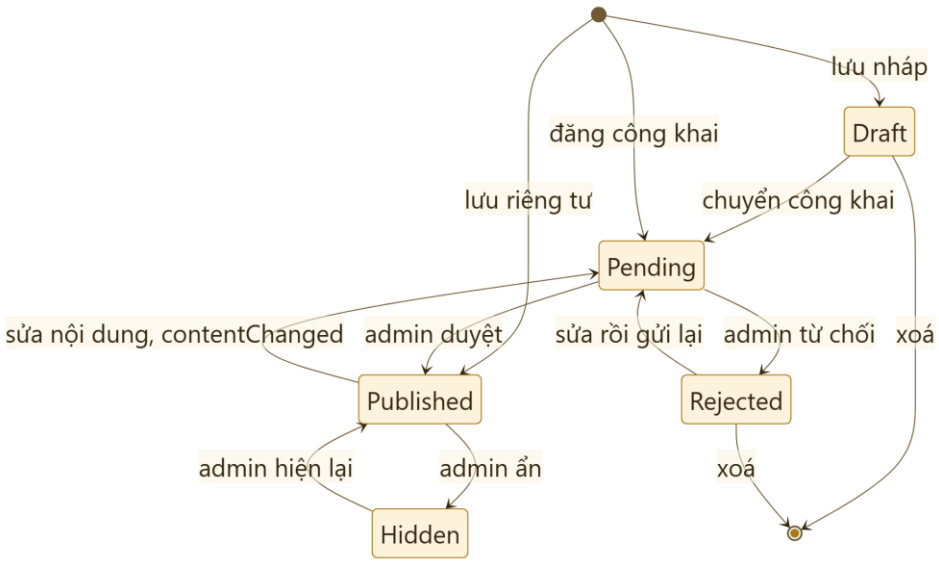
Bảng 4.4: Mức độ đầy đủ trường dữ liệu của snapshot 8.600 địa điểm

Trường	Số địa điểm	Tỉ lệ	Ghi chú
Địa chỉ	5.731	66,6 %	Một phần bổ sung bằng reverse geocoding
Loại món (cuisine)	5.500	64,0 %	Suy ra từ tag OSM và tên
Giờ mở cửa	4.292	49,9 %	Định dạng opening_hours của OSM
Điện thoại	4.141	48,2 %	Định dạng phone của OSM
Mức giá	3.450	40,1 %	Dải 1–4 theo tag price_range
Website	1.548	18,0 %	Chỉ liên kết do cộng đồng OSM ghi

Phân bố theo danh mục: restaurant 2.506, cafe 1.784, Nhà hàng 634, Đồ chay 614, Quán cà phê 587, Tiệm bánh 577, Quán ăn bình dân 570, Quán ăn vặt 550, fast_food 309, other 273, bakery 138, dessert 54.

Trạng thái xác minh: 5.150 bản ghi chưa có cờ và 3.450 unverified. Không có bản ghi nào tự nhận là đã kiểm chứng, đúng nguyên tắc xử lý dữ liệu cộng đồng.

4.4 Vòng đời một nhật ký ẩm thực



Hình 4.17: Máy trạng thái của Food Note theo quyền riêng tư và trạng thái kiểm duyệt

(Nguồn: api/index.php:1002-1070)

Hai trục trạng thái độc lập: privacy (private, public, link) và status (draft, pending, published, rejected, hidden). Quy tắc chuyển trạng thái nằm gọn trong một dòng lệnh.

```
// api/index.php:1010 – tạo mới
$state = $isDraft ? 'draft' : ($privacy === 'public' ? 'pending' :
'published');

// api/index.php:1056-1063 – sửa: chỉ quay lại hàng chờ khi nội dung thật
sự đổi
$contentChanged = $name !== $old['food_name'] || $privacy !==
$old['privacy']
                  || $restaurantId !== $old['restaurant_id'] || …;
$state = $isDraft ? 'draft'
       : ($privacy === 'public' ? ($contentChanged ? 'pending' :
$old['status']) : 'published');
```

Logic phân nhánh như sau: ghi chú riêng tư không cần duyệt vì không ai nhìn thấy; ghi chú công khai bắt buộc đi qua hàng chờ.

Khi sửa, nếu nội dung hiển thị công khai không đổi thì giữ nguyên status đã duyệt, tránh để quản trị viên duyệt lại một ghi chú chỉ mới đổi ngày ăn.

Chế độ link tạo share_token ngẫu nhiên và chỉ lưu băm.

Truy cập qua /shared/{token} (index.php:999) yêu cầu privacy='link' AND status='published', nên một ghi chú bị ẩn hoặc rút về riêng tư mất hiệu lực chia sẻ ngay lập tức.

Khi người dùng chuyển từ public sang private, dòng 1031 đặt lại status='published' và xoá share_token_hash, không còn đường vòng nào hiển thị nội dung.

4.5 Kế hoạch nhóm: giao dịch, khoá và bình chọn

Ba bài toán đúng đồng thời: mời người lạ vào kế hoạch mà không lộ danh sách thành viên, cho phép đổi phiếu nhưng chỉ được một phiếu đang có hiệu lực, và đóng bình chọn mà không nhận phiếu ghi đè lên quyết định.

```
// api/index.php:1126 – tạo kế hoạch: chủ và token mời trong một giao dịch
db()->beginTransaction();
INSERT INTO food_plans(creator_id,…,status,invite_token_hash)
VALUES(…,'open',?)
INSERT INTO food_plan_members(food_plan_id,user_id,role)
VALUES(?,?,'owner')
db()->commit();  // invite_url trả về đúng một lần
```

Mời vào kế hoạch dùng INSERT IGNORE (dòng 1129): nếu đã là thành viên thì rowCount() bằng 0 và máy chủ trả thông báo "Bạn đã là thành viên" thay vì lỗi trùng khoá.

Liên kết mời chỉ mang token, không mang mã kế hoạch, nên không thể đoán sang kế hoạch khác.

```
// api/index.php:1142 – ghi phiếu: hai thao tác trong một giao dịch có
khoá
db()->beginTransaction();
$plan = require_plan_member($planId, $user['id'], true);   // SELECT … FOR
UPDATE
if ($plan['status'] !== 'open') throw new ApiError(409, 'Bình chọn đã
đóng.');
```

```
DELETE FROM votes … WHERE food_plan_id = ? AND user_id = ?   // gỡ phiếu
cũ
INSERT INTO votes(proposal_id,user_id) VALUES(?,?)           // ghi phiếu
mới
audit(…, 'food_plan.vote', …);  db()->commit();
```

require_plan_member(..., true) thêm mệnh đề FOR UPDATE (xem index.php:177-178) để khoá hàng thành viên đang đọc.

Nhờ đó hai request đồng thời của cùng một người không tạo ra hai phiếu, và một người không chèn được phiếu vào kế hoạch vừa bị chủ chuyển sang decided.

Kiểm tra trạng thái nằm trong giao dịch chứ không phải trước đó, để trạng thái không đổi giữa lúc kiểm tra và lúc ghi.

Kết quả bình chọn không lưu thành cột tổng.

Mỗi lần đọc, truy vấn đếm trực tiếp bằng SELECT COUNT(*) và sắp proposals theo votes DESC, phá hoà theo thời gian tạo (dòng 1132).

Cách này chậm hơn một chút nhưng luôn đúng, kể cả khi thành viên bị xoá khỏi kế hoạch.

4.6 Quản lý tệp tin và phân quyền truy cập

Ảnh và video của Food Note là nội dung do người dùng tải lên – nguồn rủi ro cao nhất của một ứng dụng có cộng đồng. Dự án áp bốn lớp kiểm tra.

```
// api/index.php:1089 – kiểm tra MIME thật, không tin phần mở rộng
$mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);
$allowed = ['image/jpeg'=>'jpg', 'image/png'=>'png', 'image/webp'=>'webp',
            'image/gif'=>'gif', 'video/mp4'=>'mp4',
'video/webm'=>'webm'];
if (!isset($allowed[$mime])) throw new ApiError(415, …);
```

Lớp một: finfo đọc nội dung nhị phân để xác định MIME thật, nên tệp đổi tên từ shell.php thành shell.jpg vẫn bị từ chối.

Lớp hai: tên lưu là 48 ký tự hex do random_bytes sinh, nằm trong thư mục con theo loại (notes/, avatars/), nên không thể đoán đường dẫn và không ghi đè tệp của người khác.

Lớp ba: .htaccess chặn thực thi trong storage/.

Lớp bốn: tệp không phục vụ bằng đường dẫn tĩnh mà qua một endpoint có kiểm tra quyền.

```
// api/index.php:1110 – chống traversing và giới hạn chủ sở hữu ảnh đại
diện
if ($viewer['id'] !== $id && $viewer['role'] !== 'admin') throw new
ApiError(404, …);
$file = dirname(__DIR__) . '/storage/uploads/' . $key;
if (!is_file($file) || !str_starts_with(realpath($file) ?: '',
        realpath(dirname(__DIR__) . '/storage/uploads/avatars') ?: '!'))
throw new ApiError(404, …);
```

realpath() rồi so tiền tố là cách chống ../ kinh điển: nếu key chứa chuỗi vượt ra ngoài thư mục cho phép, kết quả realpath không còn bắt đầu bằng tiền tố hợp lệ và bị trả 404.

Chuỗi '!' ở vế phải là mã bảo hiểm, để khi realpath thất bại thì phép so sánh luôn sai.

Thông báo dùng 404 thay vì 403 để không cho người dò biết tệp có tồn tại.

Khi người dùng xoá tài khoản (index.php:1153), hệ thống lấy danh sách storage_key của toàn bộ media thuộc về họ, xoá dòng trong CSDL trong một giao dịch, rồi mới xoá tệp vật lý theo một biểu thức chính quy trắng về định dạng tên.

Thứ tự này an toàn hơn: nếu xoá tệp trước rồi CSDL lỗi thì mất dữ liệu mà không có cách lần lại.

4.7 PWA, ngoại tuyến và tối ưu tải trang

Service worker chỉ đệm "shell" – tức bộ khung ứng dụng – chứ không đệm dữ liệu. Danh sách precache gồm đúng 11 đường dẫn tĩnh (sw.js:5-17): manifest, offline.html, ba ảnh logo, ba tệp CSS và ba tệp JS.

sw.js · dòng 5–17 · mười một đường dẫn được đưa vào Cache Storage khi cài đặt

```
const STATIC_PATHS = new Set([
  '/manifest.json',
  '/offline.html',
  '/assets/img/what2eat.svg',
  '/assets/img/what2eat-192.png',
  '/assets/img/what2eat-512.png',
  '/assets/css/style.css',
  '/assets/css/auth.css',
  '/assets/css/live-data.css',
  '/assets/js/site.js',
  '/assets/js/app.js',
  '/assets/js/restaurant-map.js',
]);
```

Hình 4.18: Mười một đường dẫn được đưa vào Cache Storage khi service worker cài đặt

(Nguồn: sw.js:5-17)

sw.js · dòng 30–34 · loại trừ tuyệt đối vùng động khỏi service worker

```
self.addEventListener('fetch', (event) => {
  const request = event.request;
  const url = new URL(request.url);
  if (request.method !== 'GET' || url.origin !== self.location.origin) return;
  if (/^\/(?:api|admin|storage|includes|tiles)(?:\/|$)/i.test(url.pathname)) return;
```

Hình 4.19: Hai điều kiện loại trừ tuyệt đối vùng động trong sự kiện fetch

(Nguồn: sw.js:30-34)

Hai điều kiện này quan trọng hơn cả phần đệm.

API là nơi có phiên người dùng, nên cache có thể phục vụ dữ liệu của người khác; admin/ và storage/ chứa nội dung nhạy cảm.

Với yêu cầu điều hướng, chiến lược là network-first và rơi về offline.html khi mất mạng (sw.js:36-42); với asset tĩnh là network rồi ghi lại, chỉ đọc cache khi request thất bại (sw.js:44-48).

Tên cache what2eat-shell-v7-district-import mang số phiên bản. Sự kiện activate xoá mọi cache cùng tiền tố nhưng khác tên (sw.js:24-28) – cách nâng cấp an toàn nhất vì không để lại tệp cũ lẫn lộn.

Về tối ưu tải trang, ba cơ chế đáng nói.

Một, nạp dữ liệu bất đồng bộ với placeholder "Đang tải địa điểm…". Hai, dựng danh sách theo lô để không tạo hàng nghìn nút DOM một lần. Ba, dùng proxy tile có Cache-Control dài hạn để ảnh bản đồ được trình duyệt giữ lại mà không hỏi lại máy chủ.

4.8 Phía trình duyệt: tổ chức mã không dùng bundler

Năm tệp JavaScript, khoảng 3.000 dòng, không có bước build.

site.js (lớn nhất) chứa lớp gọi API, định dạng tiền và số đo, chuẩn hoá văn bản tiếng Việt, suy ra loại món ăn, dựng lưới thẻ và xử lý cuộn.

restaurant-map.js phụ trách Leaflet, marker, pop-up và tính khoảng cách. app.js giữ tương tác biểu mẫu và hộp thoại. create-food-note.js và dashboard chuyên biệt cho hai trang nặng.

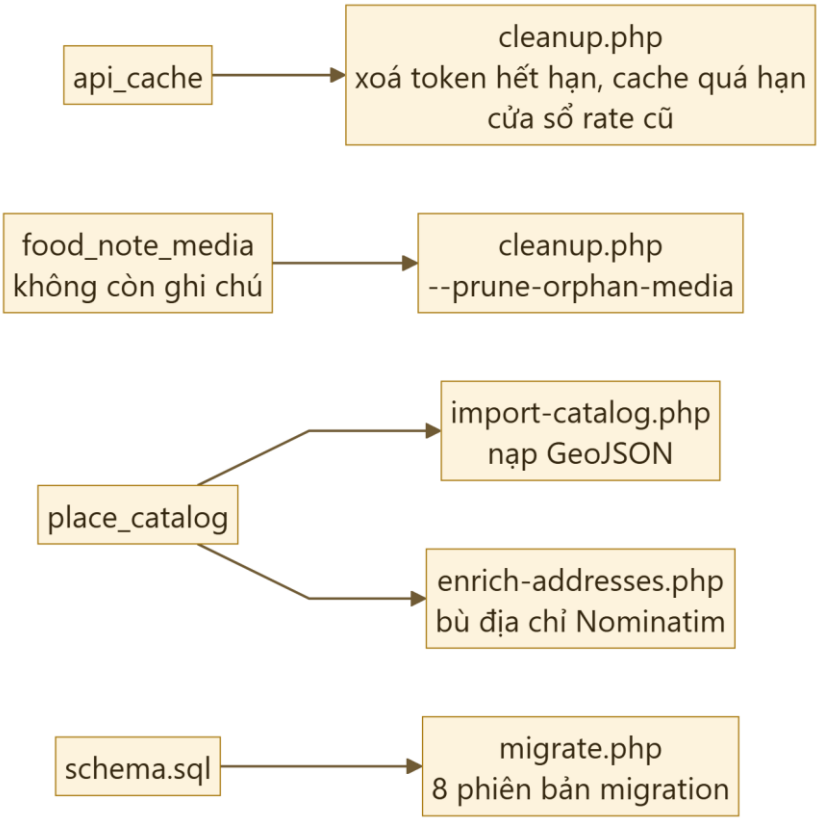
Không có bundler nên không có module thật sự giữa các tệp; dự án tránh xung đột bằng cách đặt toàn bộ trong một hàm bao (IIFE) và chỉ mở ra vài hàm cần thiết.

Cái giá phải trả là không có dependency graph và không tree-shaking. Đổi lại, mã nguồn trên máy chủ hosting trùng khớp 100% với mã trong repo, rất thuận cho việc nộp bài và chỉnh sửa nhanh.

Về khả năng truy cập, CSS tự viết chiếm phần lớn: 10 tệp, khoảng 8.300 dòng, bảng màu đặt tập trung.

Tỉ số tương phản được nâng lên mức AA trong đợt sửa ngày 04/11/2026 (tệp What2Eat-contrast-fix-2026-11-04.zip ở thư mục dự án). Cỡ chữ tối thiểu 14 px ở thân văn bản và mọi điều khiển đều có nhãn tiếng Việt.

4.9 Vận hành: migration, công cụ dòng lệnh và lịch chạy định kỳ



Hình 4.20: Luồng khởi tạo và các công cụ vận hành của dự án

(Nguồn: api/bin/*.php, scripts/*.mjs)

api/bin/migrate.php giữ tám phiên bản theo thứ tự thời gian: 20261005_initial, 20261025_sponsorships,20261025_z_session_version, 20261025_zz_place_reports, 20261025_zzz_place_quality, 20261027_google_sign_in, 20261028_completion, 20261028_activity_verification. Mỗi bước kiểm tra trạng thái hiện có (ví dụ SHOW COLUMNS FROM users LIKE 'session_version') trước khi ALTER, nên chạy lại hai lần không hỏng.

Bốn công cụ còn lại: create-admin.php dùng cơ chế upsert để tạo hoặc sửa tài khoản quản trị; import-catalog.php nạp snapshot JSON vào place_catalog trong một giao dịch.

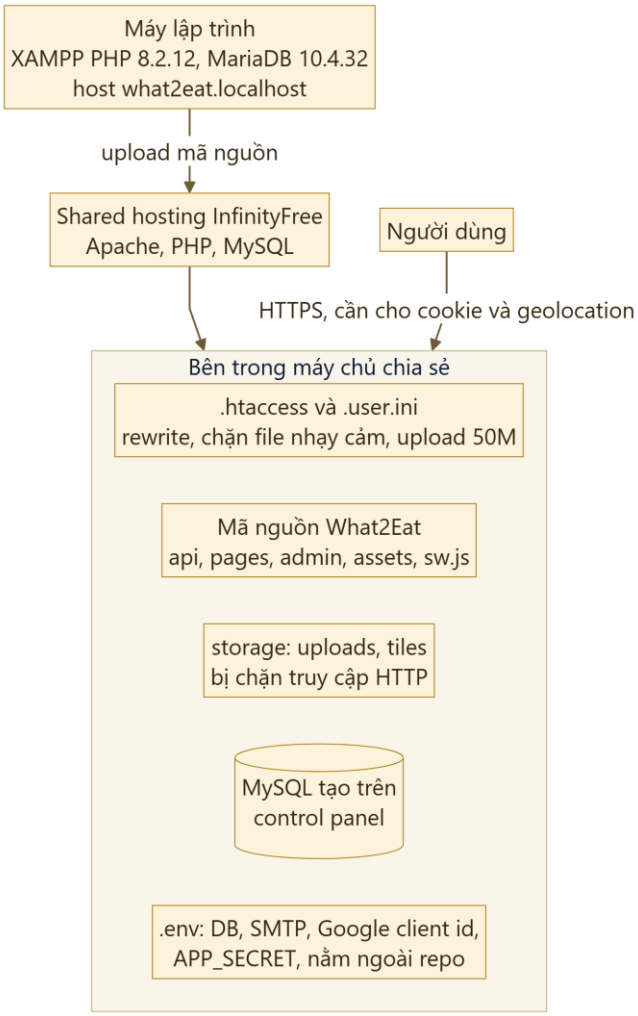
enrich-addresses.php bổ sung địa chỉ còn thiếu qua Nominatim; cleanup.php dọn rate_limits cũ hơn hai ngày và các hàng đợi hết hiệu lực.

Các script này chạy được qua cron mà không cần tương tác.

4.10 Triển khai

Nhóm chọn InfinityFree – dịch vụ hosting PHP/MySQL miễn phí – làm môi trường chạy thử công khai, vì không có chi phí, hỗ trợ sẵn PHP 8 và MySQL, cho phép dùng .htaccess và đặt biến môi trường.

Hệ thống được đẩy lên bằng cách chép toàn bộ thư mục dự án, chạy migrate một lần, cấu hình .env trên máy chủ và trỏ tên miền what2eat.click vào hosting.



Hình 4.21: Sơ đồ triển khai và các thành phần chạy trên máy chủ

(Nguồn: DEPLOYMENT.md, .htaccess)

Ba điểm phải điều chỉnh khi lên môi trường chia sẻ.

Một, bật extension PDO/MySQLi và cURL của máy chủ. Hai, trỏ W2E_ENV=production cùng W2E_BASE_URL=https://what2eat.click để cờ Secure của cookie và kiểm tra Origin hoạt động đúng. Ba, đặt quyền ghi cho storage/uploads cùng storage/cache.

Vì không kiểm soát được máy chủ, dự án không giả định có SSH hay cron hệ thống: cleanup được gọi khi có lượt truy cập thay vì chờ cron.

[Không rõ] Đăng nhập Google yêu cầu Authorized JavaScript Origin chính xác; ở môi trường chia sẻ miễn phí chưa cấu hình được client production, nên tài liệu DEPLOYMENT.md ghi rõ trạng thái này và luồng Google chỉ được kiểm chứng ở máy cục bộ.

4.11 Kiểm thử

Bốn tệp kiểm thử: tests/integration.mjs (311 dòng) cho luồng đầu-cuối, tests/google-auth.php và tests/map-location.mjs cho hai mô-đun khó, tests/random-filters.mjs cho logic chọn ngẫu nhiên và bộ lọc.

Điểm hay của integration.mjs là tính tự lập.

Nó sinh tên CSDL tạm w2e_it_<pid>_<timestamp>, tự khởi động một PHP built-in server trên cổng do hệ thống cấp (dùng net để dò cổng trống), tạo tài khoản, lấy CSRF token rồi chạy qua từng nhóm chức năng; cuối cùng xoá hẳn cơ sở dữ liệu tạm.

Không cần dữ liệu mẫu, không làm bẩn CSDL phát triển.

Dòng khẳng định cuối cùng liệt kê đúng phạm vi đã kiểm tra: "Integration checks passed: accounts, sessions, admin deletion and email reuse, place details/reports, notes, media, sponsorship, favorites, plans, profile, dashboard." – mười hai nhóm, tương ứng với các màn hình ở Chương 3.

Email trong test không gửi SMTP thật mà ghi vào storage/dev-mail.log nhờ cờ W2E_DEV_MAIL_CAPTURE, để luồng xác minh vẫn kiểm chứng được trong CI nội bộ.

4.12 Kết luận chương 4

Chương đã đi hết đường đi của dữ liệu: từ rewrite của Apache, qua router và lớp bootstrap, tới xác thực hai phương thức và nghiệp vụ có giao dịch.

Phần còn lại là khai thác OSM với bốn tầng dự phòng, quản lý tệp an toàn, đệm ngoại tuyến và các công cụ vận hành.

Những kỹ thuật đáng giá nhất không phải là công nghệ mới, mà là cách xử lý các tình huống hỏng: dịch vụ ngoài chậm, người dùng đổi ý, nội dung cần duyệt lại, phiên bị thu hồi.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được so với mục tiêu

Bảng 5.1: Đối chiếu mục tiêu đề ra ở Chương 1 với kết quả kiểm chứng được trong mã nguồn

Mục tiêu	Kết quả	Bằng chứng	Đạt
Khám phá quán ăn gần nơi ở	Bản đồ + danh sách + 4 bộ lọc + chọn ngẫu nhiên	pages/discovery.php, api/index.php:487-620	Hoàn thành
Nhật ký ẩm thực cá nhân	CRUD, nháp, ba mức riêng tư, ảnh, nhãn, ghim	api/index.php:1002-1070	Hoàn thành
Kế hoạch ăn nhóm có bình chọn	Mời bằng token, đề xuất, một phiếu/người, khoá FOR UPDATE	api/index.php:1122-1147	Hoàn thành
Thống kê cá nhân	4 chỉ số, biểu đồ 6 tháng, top món, top quán	api/index.php:1165	Hoàn thành
Nền dữ liệu quán ăn	8.600 địa điểm TP.HCM + hàng chờ kiểm duyệt	assets/data/hcm-food-places.json, place_catalog	Hoàn thành
An toàn xác thực và phiên	Argon2id, session_version, CSRF 2 lớp, 10 bộ rate limit	api/bootstrap.php:184-244	Hoàn thành
Đăng nhập Google	Tự xác minh JWT RS256, 158 dòng không thư viện	api/google-auth.php	Hoàn thành (cục bộ)
Giao diện thích ứng, PWA	20 trang, 11 đường dẫn precache, offline.html	sw.js, manifest.json	Hoàn thành
Kiểm thử tự động	12 nhóm chức năng, CSDL tạm tự sinh tự xoá	tests/integration.mjs	Hoàn thành
Triển khai công khai	Chạy trên InfinityFree với tên miền what2eat.click	Ảnh chụp Chương 3, DEPLOYMENT.md	Hoàn thành

Về khối lượng, dự án gồm 81 tệp với 29.548 dòng.

Cụ thể: 11 tệp PHP phía API (2.398 dòng), 26 tệp PHP phía trang (10.668 dòng), 12 tệp PHP quản trị (2.395 dòng), 5 tệp JavaScript (2.994 dòng), 10 tệp CSS (8.328 dòng) và 4 tệp kiểm thử (478 dòng).

Cơ sở dữ liệu có 23 bảng; API có 56 đường dẫn với 76 cặp phương thức – đường dẫn.

5.2 Hạn chế còn tồn tại

- Dữ liệu quán ăn phụ thuộc OpenStreetMap: độ đầy đủ chỉ đạt 40–67 % tuỳ trường (Bảng 4.4) và 3.450 bản ghi vẫn mang nhãn unverified.
- Ba dịch vụ ngoài (Overpass, Nominatim, tile) là máy chủ cộng đồng, không có SLA; khi bị chặn thì ứng dụng rơi về snapshot và mất khả năng cập nhật.
- Hosting miễn phí không cho phép cron hệ thống, SSH và tiến trình dài; một số bước vận hành phải chạy nhờ lượt truy cập, không phù hợp tải thật.
- Kiểm thử tự động chưa phủ phía trình duyệt; 3.000 dòng JavaScript chủ yếu được kiểm chứng thủ công qua 33 màn hình ở Chương 3.

5.3 Kết luận chung

What2Eat đã đi từ một câu hỏi thường ngày tới một hệ thống chạy thật: 20 trang giao diện, 56 đường dẫn API, 23 bảng quan hệ, 8.600 địa điểm và một bộ kiểm thử đầu-cuối tự chuẩn bị môi trường.

Giá trị lớn nhất của đề tài không nằm ở số dòng code.

Nó nằm ở chỗ đề tài buộc người làm phải giải quyết đồng thời bốn bài toán mà giáo trình tách thành bốn học phần riêng: thiết kế cơ sở dữ liệu, giao tiếp client – server, tích hợp dịch vụ bên thứ ba và bảo mật ứng dụng.

Quá trình làm dự án cũng cho thấy giới hạn thật của một nhóm hai sinh viên.

Những phần dễ kiểm chứng (mô hình dữ liệu, xác thực, kiểm duyệt) thì hoàn thiện; những phần cần hạ tầng và người dùng thật (hiệu năng khi tải lớn, chất lượng dữ liệu cộng đồng, thanh toán) thì mới đặt nền.

Nhóm xem đây là điểm nối tiếp hợp lý cho giai đoạn sau.

TÀI LIỆU THAM KHẢO

Tài liệu được trích theo thứ tự xuất hiện trong báo cáo. Các đường dẫn là trang tài liệu chính thức của công nghệ mà dự án sử dụng, kèm tài liệu nội bộ của repo.

Tài liệu tiếng Việt và tài liệu nội bộ

[1] Trường Đại học Tôn Đức Thắng. (2026). Đề tài "What2Eat – Ứng dụng web khám phá và ghi chép ẩm thực" [Tài liệu mô tả đề tài, file mô tả đề tài.txt].

[2] Nhóm tác giả What2Eat. (2026). Tài liệu thiết kế cơ sở dữ liệu (database_design.md).

[3] Nhóm tác giả What2Eat. (2026). Danh sách kiểm tra triển khai (DEPLOYMENT.md) và nhật ký hoàn thiện (COMPLETION-20261028.md).

[4] Nhóm tác giả What2Eat. (2026). Yêu cầu nhập dữ liệu OpenStreetMap (requirements-osm-import.txt).

[5] Báo cáo tiến độ đồ án What2Eat (What2Eat_BaoCaoTienDo.docx), học kỳ 1 năm học 2026–2027.

Tài liệu tiếng Anh

[6] PHP Manual. (n.d.). Password Hashing – function.password_hash. https://www.php.net/manual/en/function.password-hash.php

[7] PHP Manual. (n.d.). PHP Data Objects (PDO). https://www.php.net/manual/en/book.pdo.php

[8] PHP Manual. (n.d.). cURL functions – curl_multi_exec. https://www.php.net/manual/en/function.curl-multi-exec.php

[9] Oracle. (n.d.). MySQL 8.0 Reference Manual – InnoDB locking and foreign keys. https://dev.mysql.com/doc/refman/8.0/en/

[10] Apache Software Foundation. (n.d.). mod_rewrite – URL Rewriting Guide. https://httpd.apache.org/docs/current/rewrite/

[11] OpenStreetMap Foundation. (n.d.). Overpass API documentation. https://overpass-api.de/api_doc.html

[12] OpenStreetMap Wiki. (n.d.). Overpass QL Language Guide. https://wiki.openstreetmap.org/wiki/Overpass_API/Language_Guide

[13] OpenStreetMap Foundation. (n.d.). Nominatim usage policy. https://operations.osmfoundation.org/policies/nominatim/

[14] OpenStreetMap Foundation. (n.d.). Tile usage policy. https://operations.osmfoundation.org/policies/tiles/

[15] Geofabrik GmbH. (n.d.). OpenStreetMap data for download – Vietnam. https://download.geofabrik.de/asia/vietnam.html

[16] Beutner, A., et al. (2023). Leaflet 1.9 API documentation. https://leafletjs.com/reference.html

[17] Twitter, Inc. (2024). Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

[18] Chart.js contributors. (2024). Chart.js 4 documentation. https://www.chartjs.org/docs/latest/

[19] Google LLC. (2024). Verify the Google ID token on your server side. https://developers.google.com/identity/gsi/web/guides/verify-google-id-token

[20] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519

[21] Jones, M., & Skora, S. (2015). JSON Web Key (JWK), RFC 7517. https://datatracker.ietf.org/doc/html/rfc7517

[22] Biryukov, A., et al. (2015). Argon2: the Password Hashing Competition winner. https://www.password-hashing.net/

[23] Laurie, B., et al. (2022). Argon2 Memory-Hard Function for Password Hashing, RFC 9106. https://datatracker.ietf.org/doc/html/rfc9106

[24] OWASP Foundation. (2024). Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[25] OWASP Foundation. (2024). CSRF Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[26] OWASP Foundation. (2024). File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[27] Mozilla Foundation. (2024). Service Worker API. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

[28] Mozilla Foundation. (2024). Web App Manifest. https://developer.mozilla.org/en-US/docs/Web/Manifest

[29] Unicode Consortium. (2024). Unicode Normalization Forms, UTR #15. https://unicode.org/reports/tr15/

[30] PHPMailer contributors. (2024). PHPMailer 7 documentation. https://github.com/PHPMailer/PHPMailer

[31] Composer. (2024). Dependency Manager for PHP. https://getcomposer.org/doc/

[32] InfinityFree. (2024). Free PHP web hosting documentation. https://www.infinityfree.com/

[33] Wikipedia. (2024). Haversine formula. https://en.wikipedia.org/wiki/Haversine_formula